DeepSeek开发指南

1. DeepSeek介绍

DeepSeek大模型基于深度学习技术，特别是Transformer架构，这是一种在自然语言处理领域非常成功的神经网络结构。Transformer架构中的自注意力机制使得模型能够捕捉文本中的长距离依赖关系，从而更好地理解上下文信息。

DeepSeek大模型相比其他模型具有以下几个显著的优势：

1）性能卓越

DeepSeek大模型在各种自然语言处理任务中表现出了卓越的性能，如文本生成、问答系统、机器翻译等。它能够生成高质量、有创造性的文本，并且能够准确地回答各种复杂的问题。

2）效率高

DeepSeek大模型在训练和推理过程中都表现出了较高的效率。它采用了先进的优化算法和并行计算技术，能够加速训练过程，提高模型的收敛速度。同时，在推理过程中，DeepSeek大模型也能够快速地生成响应，满足实时应用的需求。

3）成本低

相比其他大型模型，DeepSeek大模型在训练和部署成本方面具有一定的优势。它采用了更加高效的算法和架构设计，减少了计算资源的消耗。同时，DeepSeek大模型也提供了多种部署方式，如云端部署和边缘部署，以满足不同场景下的需求。

4）可扩展性强

DeepSeek大模型具有良好的可扩展性，能够根据不同的任务需求进行定制和优化。它支持多种输入和输出格式，并且能够与其他系统进行集成，实现更加复杂的应用场景。

2. 本地化部署

详见视频https://www.bilibili.com/video/BV15zNGeYE8M?t=326.4

3. 基本开发

2.1 准备工作

- 第一步：安装openai开发库

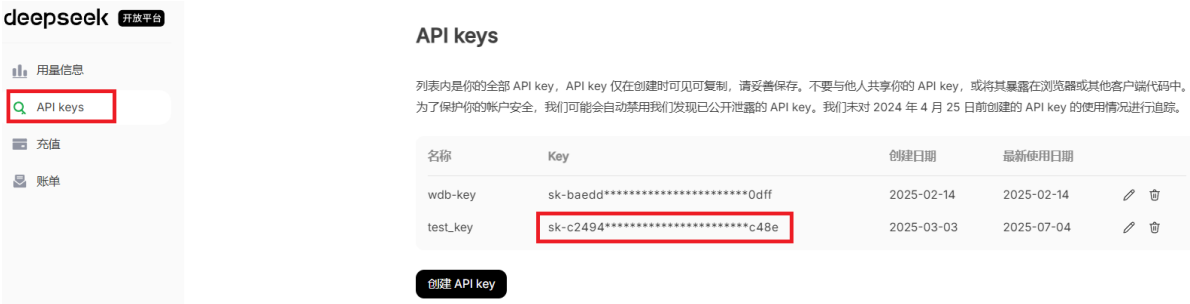
  如果使用conda解释器，先创建虚拟环境：

  ```
  conda create -n llm python=3.10
  # 创建完成之后
  conda activate llm
  ```

  安装openai开发库（如果使用原生Python，直接执行下面的指令安装）：

  ```
  pip install openai
  ```

- 第二步：到DeepSeek开放平台下注册账号，地址 https://platform.deepseek.com/usage
- 第三步：进入个人中心，充费
- 第四步：创建key，并复制，保存



- 第四步：PyCharm下创建项目，安装openai开发库
- 第五步：新建global_var.py，将第三步保存的key写入文件中。内容如下：

```
ds_key = '这里改成你的key'
```

2.2 调用本地模型

```
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/', # 本机或局域网地址
    api_key='ollama',
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            'role': 'user',
            'content': '哪吒这个词是什么意思，帮我详细解释一下',
        }
    ],
    model = "deepseek-r1:7b",
    temperature=0.0,  # 可以根据需要调整温度值，决定生成的随机性程度
)

# 打印结果
print("模型返回的内容：")
print(chat_completion.choices[0].message.content)
```

2.3 远程调用

```
from openai import OpenAI
from global_var import *

client = OpenAI(api_key=ds_key, base_url="https://api.deepseek.com")

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        # system消息：向模型传递指令（如设定角色、要求输出格式）
        {"role": "system", "content": "You are a helpful assistant"},
```

```python
        # user消息：用户当前输入内容
        {"role": "user", "content": "你好，帮我详细解释一下哪吒是什么意思"},
    ],
    stream=False # 是否流式返回结果（False表示一次性返回完整响应）
)

print(response.choices[0].message.content)
```

2.4 对话补全

对话补全指给出开头、结尾部分，对中间部分进行补写。

```python
from openai import OpenAI
from global_var import *

# user should set `base_url="https://api.deepseek.com/beta"` to use this feature.
client = OpenAI(api_key=ds_key,
                base_url="https://api.deepseek.com/beta")
response = client.completions.create(
    model="deepseek-chat",
    prompt="视频检测讲义\n 第一节 视频检测概述",
    suffix="综上所述，视频检测就是从视频中检测出所感兴趣的目标物体，并标记出类别和位置。",
    max_tokens=512)
print(response.choices[0].text)
```

2.5 对话前缀续写

```python
from openai import OpenAI
from global_var import *

client = OpenAI(api_key=ds_key,
                base_url="https://api.deepseek.com/beta")

messages = [
    {"role": "user", "content": "Please write quick sort code"},
    {"role": "assistant", "content": "```python\n", "prefix": True}
]
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=messages,
    stop=["```"],
)
print(response.choices[0].message.content)
```

2.6 JSON Output

在很多场景下，用户需要让模型严格按照 JSON 格式来输出，以实现输出的结构化，便于后续逻辑进行解析。DeepSeek 提供了 JSON Output 功能，来确保模型输出合法的 JSON 字符串。

```python
import json
from openai import OpenAI
from global_var import *

```

```python
client = OpenAI(api_key=ds_key, base_url="https://api.deepseek.com")

system_prompt = """
用户将提供一些测试文本。请解析“问题”和“答案”，并以JSON格式输出。
例如:
世界上最高的山是哪座？珠穆朗玛峰。

示例json输出:
{
    "question": "世界上最高的山是哪座？",
    "answer": "珠穆朗玛峰"
}
"""

user_prompt = "世界上最长的河是哪条河流.尼罗河"

messages = [{"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}]

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=messages,
    response_format={
        'type': 'json_object'
    }
)

print(json.loads(response.choices[0].message.content))
```

2.7 多轮对话

```python
from openai import OpenAI
from global_var import *

client = OpenAI(api_key=ds_key, base_url="https://api.deepseek.com")

# Round 1
messages = [
    {
        "role": "user",
        "content": "世界上最高的山峰是哪座山?"
    }
]
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=messages
)

messages.append(response.choices[0].message) # 添加到messages
print(f"Messages Round 1:", response.choices[0].message.content)

# Round 2
messages.append({"role": "user", "content": "这座山属于哪个国家或哪几个国家?"})
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=messages
)
```

```python

messages.append(response.choices[0].message)
print(f"\n\nMessages Round 2:", response.choices[0].message.content)

print("")

print(messages)
```

4. Function Calling

4.1 什么是Function Calling

Function Calling（函数调用）是大语言模型（LLM）的一项重要能力，它允许AI模型在理解用户请求后，不仅生成文本回复，还能识别出需要调用**外部工具或API**来完成特定任务，并生成结构化的函数调用请求。简单来说，Function Calling就像是给LLM配备了一个"工具箱"。当用户提出复杂需求时，模型不再局限于文本生成，而是能够判断："这个问题需要调用哪个工具？需要传递什么参数？"然后生成标准的函数调用格式，由外部系统实际执行相关操作。

4.2 为什么需要Function Calling

1）突破模型固有局限

大语言模型虽然知识渊博，但存在固有局限：知识可能过时、无法访问实时数据、缺乏专业计算能力。Function Calling通过连接外部工具，完美解决了这些问题。

2）实现真正的人机协作

用户可以用自然语言描述复杂需求，模型将其转化为精确的技术操作。比如用户说"帮我订明天北京到上海的机票"，模型可以调用机票预订API，而不是仅仅给出文字建议。

3）提升应用实用性

在实际应用中，纯文本回复往往不够。Function Calling让LLM能够执行实际操作：查询数据库、调用计算工具、操作外部系统等，大大提升了AI应用的实用价值。

4）保证输出结构化

传统文本输出需要复杂的解析，而Function Calling产生标准化的JSON格式，便于程序自动化处理，提高了系统集成的可靠性。

4.3 Function Calling如何工作

工作流程分为四个关键步骤：

- **意图识别**：模型分析用户查询，判断是否需要调用外部函数
- **函数选择**：从预定义的函数库中选择最合适的工具
- **参数生成**：根据查询内容提取并生成准确的函数参数
- **结果整合**：将函数执行结果融入最终回复中

4.4 什么时候用Function Calling

当大模型需要获得外部知识、外部能力时，使用Function Calling，例如：调用航班查询、酒店预订、天气API、连接数据库获取实时业务数据、触发OA系统审批流程、生成分析报告、连接文件系统、调用专业计算工具、连接学术数据库、调用科学计算软件.....

Function Calling代表了LLM从"对话工具"向"行动代理"的演进，通过将语言理解与实际行动能力结合，为AI应用开辟了全新的可能性。这项技术正在成为构建下一代智能系统的核心能力，让AI不仅能够回答问题，更能够解决问题。

4.4.1 示例一

```python
from openai import OpenAI
from global_var import *


def send_messages(messages):
    response = client.chat.completions.create(
        model="deepseek-chat",
        messages=messages,
        tools=tools
    )
    return response.choices[0].message


client = OpenAI(
    api_key=ds_key,
    base_url="https://api.deepseek.com",
)

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather of an location, the user shoud supply a location first",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "The city and state, e.g. San Francisco, CA",
                    }
                },
                "required": ["location"]
            },
        }
    },
]

messages = [
    {
        "role": "user",
        "content": "How's the weather in Hangzhou?"
    }
]
message = send_messages(messages)
print(f"User>\t {messages[0]['content']}")

tool = message.tool_calls[0]
messages.append(message)

print(tool)

```

```
messages.append({"role": "tool", "tool_call_id": tool.id, "content": "24℃"})
message = send_messages(messages)
print(f"Model>\t {message.content}")
```

4.4.2 示例二

```
import json
from math import *
from dotenv import find_dotenv, load_dotenv
import requests
from global_var import *

# DeepSeek API配置
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
DEEPSEEK_API_KEY = ds_key

def print_json(data):
    """
    打印参数。如果参数是有结构的（如字典或列表），则以格式化的 JSON 形式打印；
    否则，直接打印该值。
    """
    if hasattr(data, "model_dump_json"):
        data = json.loads(data.model_dump_json())

    if isinstance(data, (list)):  # 如果data为列表
        for item in data:  # 打印每个元素
            print_json(item)
    elif isinstance(data, (dict)):  # 字典
        print(json.dumps(data, indent=4, ensure_ascii=False))
    else:  # 其它直接打印
        print(data)


def get_completion(messages, model="deepseek-chat"):
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",
        "Content-Type": "application/json"
    }

    payload = {
        "model": model,
        "messages": messages,
        "tools": [
            {
                "type": "function",
                "function": {
                    "name": "sum",
                    "description": "加法器，计算一组数的和，只能运用于加法操作",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "numbers": {"type": "array", "items": {"type": "number"}}
                        },
                        "required": ["numbers"]
                    },
```

```
                },
            }
        ]
    }

    response = requests.post(DEEPSEEK_API_URL, headers=headers, json=payload)
    response.raise_for_status()
    return response.json()["choices"][0]["message"]


# 测试提示词
prompt = "桌上有 2 个苹果，四个桃子和 3 本书，一共有几个水果？"

messages = [
    {"role": "system", "content": "你是一个数学家"},
    {"role": "user", "content": prompt},
]
response = get_completion(messages)
print(response)

messages.append(response)  # 把大模型的回复加入到对话历史中

print("================= DeepSeek回复 =================")
print_json(response)

# 如果返回的是函数调用结果，则打印出来
if response.get("tool_calls") is not None:
    # 是否要调用 sum
    tool_call = response["tool_calls"][0]
    if tool_call["function"]["name"] == "sum":
        # 调用 sum
        args = json.loads(tool_call["function"]["arguments"])
        result = sum(args["numbers"])
        print("=====函数返回=====")
        print(result)

        # 把函数调用结果加入到对话历史中
        messages.append(
            {
                "tool_call_id": tool_call["id"],  # 用于标识函数调用的 ID
                "role": "tool",
                "name": "sum",
                "content": str(result),  # 数值 result 必须转成字符串
            }
        )

        # 再次调用大模型
        print("=====最终回复=====")
        print(get_completion(messages)["content"])
```

4.4.3 示例三

```
import json
from dotenv import find_dotenv, load_dotenv
import requests
import os
```

```python
from global_var import *

# DeepSeek API配置
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
DEEPSEEK_API_KEY = ds_key


def print_json(data):
    """
    打印参数。如果参数是有结构的（如字典或列表），则以格式化的 JSON 形式打印；
    否则，直接打印该值。
    """
    if hasattr(data, "model_dump_json"):
        data = json.loads(data.model_dump_json())

    if isinstance(data, (list)):
        for item in data:
            print_json(item)
    elif isinstance(data, (dict)):
        print(json.dumps(data, indent=4, ensure_ascii=False))
    else:
        print(data)


def get_completion(messages, model="deepseek-chat"):
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",
        "Content-Type": "application/json"
    }

    payload = {
        "model": model,
        "messages": messages,
        "temperature": 0,
        "seed": 1024,
        "tool_choice": "auto",
        "tools": [
            {
                "type": "function",
                "function": {
                    "name": "get_location_coordinate",
                    "description": "根据POI(Point of Interest)名称，获得POI的经纬度坐标",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "location": {
                                "type": "string",
                                "description": "POI名称，必须是中文",
                            },
                            "city": {
                                "type": "string",
                                "description": "POI所在的城市名，必须是中文",
                            },
                        },
                        "required": ["location", "city"],
                    },
                },
```

```
            },
            {
                "type": "function",
                "function": {
                    "name": "search_nearby_pois",
                    "description": "搜索给定坐标附近的POI",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "longitude": {
                                "type": "string",
                                "description": "中心点的经度",
                            },
                            "latitude": {
                                "type": "string",
                                "description": "中心点的纬度",
                            },
                            "keyword": {
                                "type": "string",
                                "description": "目标POI的关键字",
                            },
                        },
                        "required": ["longitude", "latitude", "keyword"],
                    },
                },
            },
        ],
    }

    response = requests.post(DEEPSEEK_API_URL, headers=headers, json=payload)
    response.raise_for_status()
    return response.json()["choices"][0]["message"]


amap_key = "bac2dcef25b053ea881ddc0862647407"  # 高德地图密钥


def get_location_coordinate(location, city):
    # 获取高德地图的位置坐标
    url = f"https://restapi.amap.com/v5/place/text?key={amap_key}&keywords={location}&region={city}"
    print(url)
    r = requests.get(url)
    result = r.json()
    if "pois" in result and result["pois"]:
        return result["pois"][0]
    return None


def search_nearby_pois(longitude, latitude, keyword):
    # 搜索坐标附近的关键词poi
    url = f"https://restapi.amap.com/v5/place/around?key={amap_key}&keywords={keyword}&location={longitude},{latitude}"
    print(url)
    r = requests.get(url)
    result = r.json()

```

```python
    ans = ""
    if "pois" in result and result["pois"]:  # 如果有结果且pois在响应结果中
        for i in range(min(3, len(result["pois"]))):
            name = result["pois"][i]["name"]
            address = result["pois"][i]["address"]
            distance = result["pois"][i]["distance"]
            ans += f"{name}\n{address}\n距离：{distance}米\n\n"
    return ans


prompt = "我想在北京五道口附近喝咖啡，给我推荐几个"
# prompt = "我到北京出差，给我推荐三里屯的酒店，和五道口附近的咖啡"

messages = [
    {"role": "system", "content": "你是一个地图通，你可以找到任何地址。"},
    {"role": "user", "content": prompt},
]
response = get_completion(messages)
messages.append(response)  # 把大模型的回复加入到对话中
print("========================DeepSeek回复1========================")
print_json(response)

i = 0

while response.get("tool_calls") is not None:
    # 支持多个函数调用
    print("\n第%d轮函数调用" % (i + 1))
    for tool_call in response["tool_calls"]:
        args = json.loads(tool_call["function"]["arguments"])
        print("函数参数展开：")
        print_json(args)

        if tool_call["function"]["name"] == "get_location_coordinate":
            print("Call: get_location_coordinate")
            result = get_location_coordinate(**args)
        elif tool_call["function"]["name"] == "search_nearby_pois":
            print("Call: search_nearby_pois")
            result = search_nearby_pois(**args)

        print("\n========================函数返回========================")
        print_json(result)

        messages.append(
            {
                "tool_call_id": tool_call["id"],  # 用于标识函数调用的 ID
                "role": "tool",
                "name": tool_call["function"]["name"],
                "content": str(result),  # 数值result 必须转成字符串
            }
        )
    i += 1

    response = get_completion(messages)
    print("========================DeepSeek回复2========================")
    messages.append(response)  # 把大模型的回复加入到对话中

```

```
print("=========================最终回复=========================")
print(response["content"])

print("\n=========================对话历史=========================")
print_json(messages)
```

4.4.4 示例四

```
import json
from math import *
from dotenv import find_dotenv, load_dotenv
import requests
import os
import sqlite3
from global_var import *

# DeepSeek API配置
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
DEEPSEEK_API_KEY = ds_key


def print_json(data):
    """
    打印参数。如果参数是有结构的（如字典或列表），则以格式化的JSON形式打印；
    否则，直接打印该值。

    参数:
        data: 要打印的数据，可以是任何类型
    """
    # 如果数据有model_dump_json方法（通常是Pydantic模型），则转换为JSON字符串再解析为字典
    if hasattr(data, "model_dump_json"):
        data = json.loads(data.model_dump_json())

    # 如果是列表，递归打印每个元素
    if isinstance(data, (list)):
        for item in data:
            print_json(item)
    # 如果是字典，以格式化的JSON形式打印
    elif isinstance(data, (dict)):
        print(json.dumps(data, indent=4, ensure_ascii=False))
    # 其他类型直接打印
    else:
        print(data)


def get_sql_completion(messages, model="deepseek-chat"):
    """
    调用DeepSeek API获取SQL查询补全结果

    参数:
        messages: 对话消息列表，包含角色和内容
        model: 使用的DeepSeek模型名称，默认为"deepseek-chat"

    返回:
        DeepSeek API的响应消息
    """
```

```python
    # 设置API请求头
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",  # Bearer令牌认证
        "Content-Type": "application/json"  # 请求内容类型为JSON
    }

    # 构建API请求负载
    payload = {
        "model": model,  # 指定使用的模型
        "messages": messages,  # 对话历史消息
        "temperature": 0,  # 温度参数为0，确保输出确定性
        "tools": [  # 定义可用的函数工具
            {
                "type": "function",
                "function": {
                    "name": "ask_database",  # 函数名称
                    "description": "Use this function to answer user questions about business. \
                            Output should be a fully formed SQL query.",  # 函数描述
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "query": {
                                "type": "string",
                                "description": f"""
                            SQL query extracting info to answer the user's question.
                            SQL should be written using this database schema:
                            {database_schema_string}
                            The query should be returned in plain text, not in JSON.
                            The query should only contain grammars supported by SQLite.
                            """,  # SQL查询的详细描述，包含数据库模式信息
                            }
                        },
                        "required": ["query"],  # 必需的参数
                    },
                },
            }
        ],
    }

    # 发送POST请求到DeepSeek API
    response = requests.post(DEEPSEEK_API_URL, headers=headers, json=payload)
    response.raise_for_status()  # 如果请求失败则抛出异常

    # 返回API响应中的第一条消息
    return response.json()["choices"][0]["message"]


# 描述数据库表结构
database_schema_string = """
CREATE TABLE orders (
    id INT PRIMARY KEY NOT NULL, -- 主键，不允许为空
```

```
    customer_id INT NOT NULL, -- 客户ID，不允许为空
    product_id STR NOT NULL, -- 产品ID，不允许为空
    price DECIMAL(10,2) NOT NULL, -- 价格，不允许为空
    status INT NOT NULL, -- 订单状态，整数类型，不允许为空。0代表待支付，1代表已支付，2代表已退款
    create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- 创建时间，默认为当前时间
    pay_time TIMESTAMP -- 支付时间，可以为空
);
"""

# 创建SQLite内存数据库连接
conn = sqlite3.connect(":memory:")  # 内存存储模式，程序结束后数据销毁
cursor = conn.cursor()  # 创建游标对象用于执行SQL语句

# 创建orders表
cursor.execute(database_schema_string)

# 插入5条明确的模拟记录
mock_data = [
    (1, 1001, "TSHIRT_1", 50.00, 0, "2023-10-12 10:00:00", None),  # 待支付订单
    (2, 1001, "TSHIRT_2", 75.50, 1, "2023-10-16 11:00:00", "2023-08-16 12:00:00"),  # 已支付订单
    (3, 1002, "SHOES_X2", 25.25, 2, "2023-10-17 12:30:00", "2023-08-17 13:00:00"),  # 已退款订单
    (4, 1003, "HAT_Z112", 60.75, 1, "2023-10-20 14:00:00", "2023-08-20 15:00:00"),  # 已支付订单
    (5, 1002, "WATCH_X001", 90.00, 0, "2023-10-28 16:00:00", None),  # 待支付订单
]

# 调用SQL语句执行插入操作
for record in mock_data:
    cursor.execute(
        """
    INSERT INTO orders (id, customer_id, product_id, price, status, create_time, pay_time)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,  # 使用参数化查询防止SQL注入
        record,  # 传入参数元组
    )

# 提交事务，确保数据持久化
conn.commit()


def ask_database(query):
    """
    执行SQL查询并返回结果

    参数:
        query: SQL查询语句

    返回:
        查询结果记录列表
    """
    cursor.execute(query)  # 执行SQL查询
```

```python
    records = cursor.fetchall()  # 获取所有查询结果
    return records


# 用户查询提示词示例
# prompt = "10月的销售额"  # 查询10月份销售额
# prompt = "统计每月每件商品的销售额"  # 按月统计商品销售额
prompt = "哪个用户消费最高？消费多少？"  # 查询消费最高的用户及金额

# 初始化对话消息
messages = [
    {"role": "system", "content": "基于 order 表回答用户问题"},  # 系统角色设置上下文
    {"role": "user", "content": prompt},  # 用户查询
]

# 获取DeepSeek的初始响应
response = get_sql_completion(messages)

# 确保响应内容不为None
if response.get("content") is None:
    response["content"] = ""

# 将响应添加到对话历史中
messages.append(response)

print("====Function Calling====")
print_json(response)  # 打印函数调用响应

# 检查是否有函数调用请求
if response.get("tool_calls") is not None:
    # 获取第一个函数调用
    tool_call = response["tool_calls"][0]

    # 检查是否为ask_database函数调用
    if tool_call["function"]["name"] == "ask_database":
        # 解析函数参数
        arguments = tool_call["function"]["arguments"]
        args = json.loads(arguments)

        print("====SQL====")
        print(args["query"])  # 打印生成的SQL查询

        # 执行SQL查询
        result = ask_database(args["query"])

        print("====DB Records====")
        print(result)  # 打印数据库查询结果

        # 将函数调用结果添加到对话历史中
        messages.append(
            {
                "tool_call_id": tool_call["id"],  # 函数调用ID，用于关联响应
                "role": "tool",  # 角色为tool，表示函数执行结果
                "name": "ask_database",  # 函数名称
                "content": str(result),  # 函数执行结果（转换为字符串）
            }
        )
```

```

        # 再次调用DeepSeek API，让模型基于函数结果生成最终回复
        response = get_sql_completion(messages)

        print("====最终回复====")
        print(response["content"])  # 打印最终回复内容
```

4.5 注意事项

1）安全性问题
Function Calling本质上赋予了LLM执行操作的能力，这带来了显著的安全风险。必须严格执行最小权限原则，仅开放必要的函数权限。例如：只读函数与写入函数要严格区分、敏感操作（如删除、支付）需要额外授权验证、设置操作频率限制防止滥用、所有函数参数必须经过严格验证，防止注入攻击等。

2）可靠性问题

调用外部函数，必须包含完善的错误处理，包括设置函数调用超时时间、实现重试机制但避免无限循环、提供有意义的错误信息但避免泄露敏感细节。当函数调用失败时，应有备用方案，例如：返回缓存数据、提供替代性文本回答、引导用户通过其他方式完成操作等。

3）性能问题

外部函数调用会增加响应时间，需要设置合理的超时阈值，也可以考虑异步执行耗时操作或优先使用本地函数减少网络延迟。

4）调用成本
Function Calling可能产生额外成本，需要监控API调用频率和费用，实施缓存策略减少重复调用，批量处理相关请求减少调用次数等。

5）用户体验问题

Function Calling会导致系统变慢，用户应该了解AI正在执行操作，明确告知将执行什么函数。对于敏感操作，需要请求用户确认，提供操作执行状态反馈。

5. RAG

5.1 什么是RAG

RAG（Retrieval Augmented Generation，检索增强生成） 是一种将信息检索与大语言模型生成能力相结合的前沿技术。它通过从外部知识源实时检索相关信息，并将这些信息作为上下文提供给LLM，从而生成更加准确、及时和可靠的回答。

与传统LLM仅依赖训练时学到的静态知识不同，RAG让模型具备了"查阅参考资料"的能力。当用户提出问题后，RAG系统首先从数据库、文档库或网络等知识源中检索最相关的信息，然后将这些信息与用户问题一起交给LLM，指导模型基于最新、最准确的信息生成回答。

RAG就像是给LLM配备了一个"智能助理"——助理负责快速查找相关资料，LLM则专注于理解和整合这些信息，最终给出高质量的回答。这种分工协作的模式显著提升了AI系统的实用性和可靠性。

5.2 RAG用在哪里

1）解决LLM的固有局限性

大语言模型存在知识截止日期、可能产生幻觉（编造信息）、以及无法访问私有或最新数据等问题。RAG通过实时检索机制完美弥补了这些缺陷，让模型能够基于最新、最准确的信息进行回答。

2）提升回答的准确性和可信度

通过提供确切的参考信息作为生成依据，RAG大大减少了模型编造信息的可能性。每个回答都有据可依，显著提高了输出的可靠性和可信度。

3）保护数据隐私和降低成本

企业可以将敏感数据保存在私有数据库中，仅通过检索方式提供给模型，既保护了数据安全，又避免了重新训练模型的高昂成本。

4）实现个性化服务

RAG允许系统根据不同用户的特定需求，从个性化数据源中检索信息，从而提供更加精准和个性化的服务。

5.3 RAG的工作流程

RAG的工作流程包含三个核心阶段：

1）检索阶段（Retrieval）

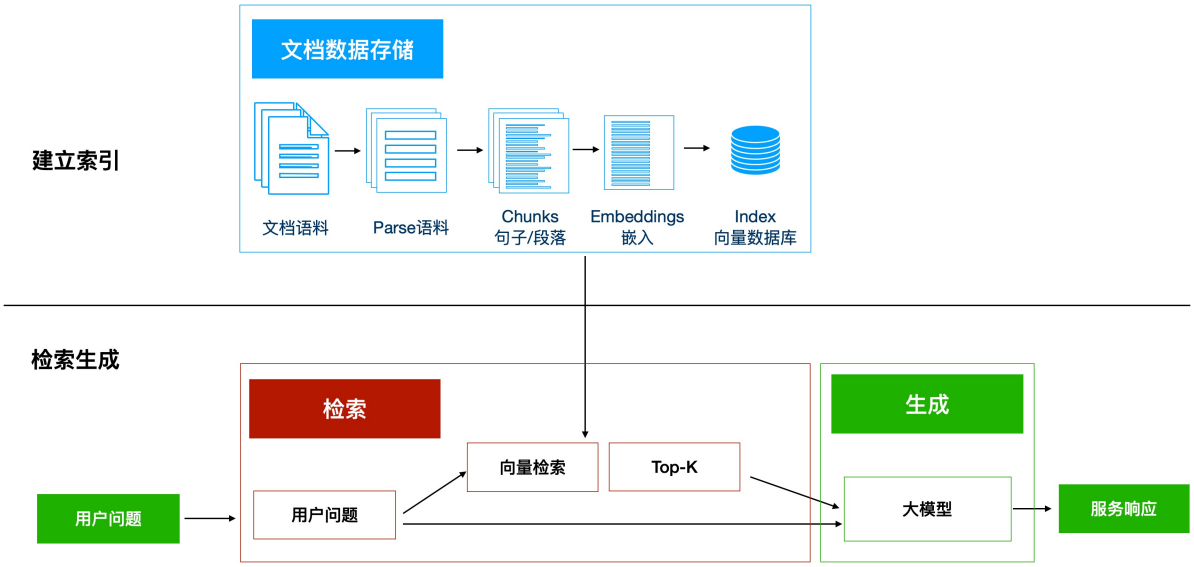
- 将用户查询转换为向量表示
- 在向量数据库中进行相似度搜索
- 返回最相关的文档片段

2）增强阶段（Augmentation）

- 将检索到的信息与用户问题结合
- 构建包含上下文的提示词
- 确保信息的相关性和完整性

3）生成阶段（Generation）

- LLM基于增强后的提示词生成回答
- 模型引用检索到的信息进行论证
- 输出最终的自然语言回复



RAG技术实现关键点：

- 向量化：使用embedding模型将文本转换为数值向量
- 相似度计算：通过余弦相似度等算法找到最相关文档
- 提示工程：精心设计提示词模板，指导模型正确使用检索信息

5.3 简单RAG示例

示例一

```python
# RAG(Retrieval Augmented Generation)示例 - 基于DeepSeek实现
# RAG是一种结合检索和生成的AI技术，先用检索器找到相关信息，再用生成器基于这些信息回答问题

import json
import os
import requests  # 导入requests库用于HTTP请求
from dotenv import load_dotenv, find_dotenv
from global_var import *

# DeepSeek API配置
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
DEEPSEEK_API_KEY = ds_key


def get_completion(prompt, model="deepseek-chat"):
    """
    调用DeepSeek API获取对话补全结果

    参数:
        prompt: 用户输入的提示词文本
        model: 使用的DeepSeek模型名称，默认为"deepseek-chat"

    返回:
        DeepSeek API的回复内容
    """
    # 构建消息列表，包含用户角色和内容
    messages = [{"role": "user", "content": prompt}]

    # 设置API请求头
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",  # Bearer令牌认证
        "Content-Type": "application/json"  # 请求内容类型为JSON
    }

    # 构建API请求负载
    payload = {
        "model": model,  # 指定使用的模型
        "messages": messages,  # 对话内容
        "temperature": 0,  # 模型输出的随机性，0表示随机性最小（最确定性）
    }

    try:
        # 发送POST请求到DeepSeek API
        response = requests.post(DEEPSEEK_API_URL, headers=headers, json=payload)
        response.raise_for_status()  # 如果请求失败则抛出异常

        # 解析JSON响应
        response_data = response.json()
        print("DeepSeek API响应:", response_data)

        # 返回第一条选择的消息内容
        return response_data["choices"][0]["message"]["content"]
```

```python

    except requests.exceptions.RequestException as e:
        # 处理网络请求错误
        print(f"API请求错误: {e}")
        return "抱歉，API请求失败，请稍后重试。"
    except KeyError as e:
        # 处理响应格式错误
        print(f"响应格式错误: {e}")
        return "抱歉，API响应格式异常。"


def build_prompt(prompt_template, **kwargs):
    """
    构建动态提示词，将模板中的变量替换为实际值

    参数:
        prompt_template: 提示词模板字符串，包含__变量名__格式的占位符
        **kwargs: 关键字参数，用于替换模板中的变量

    返回:
        替换后的完整提示词字符串
    """
    prompt = prompt_template  # 初始化提示词为模板

    # 遍历所有关键字参数
    for k, v in kwargs.items():
        if isinstance(v, str):  # 如果值是字符串类型
            val = v
        elif isinstance(v, list) and all(isinstance(elem, str) for elem in v):  # 如果值是字符串列表
            val = '\n'.join(v)  # 用换行符连接列表中的所有字符串
        else:  # 其他数据类型
            val = str(v)  # 转换为字符串

        # 替换模板中的占位符（格式：__变量名__）
        placeholder = f"__{k.upper()}__"  # 构造占位符，如__INFO__
        prompt = prompt.replace(placeholder, val)  # 执行替换

    return prompt


# RAG提示词模板
# 这个模板指导模型如何基于检索到的信息回答问题
prompt_template = """
你是一个问答机器人。
你的任务是根据下述给定的已知信息回答用户问题。
确保你的回复完全依据下述已知信息。不要编造答案。
如果下述已知信息不足以回答用户的问题，请直接回复"我无法回答您的问题"。

已知信息:
__INFO__

用户问:
__QUERY__

请用中文回答用户问题。
"""

```

```python
# 用户查询问题
user_query = "how many parameters does llama 2 have?"

# 1. 检索阶段 (Retrieval)
# 在实际应用中，这里会调用搜索引擎或向量数据库检索相关信息
# search_results = search(user_query, 2)  # 实际检索函数调用

# 模拟检索结果 - 这里使用硬编码的检索结果作为示例
search_results = """
Llama 2 comes in a range of parameter sizes—7B, 13B, and 70B—as well as pretrained and fine-tuned variations.1. Llama 2, an updated version of Llama 1, trained on a new mix of publicly available data. We also increased the size of the pretraining corpus by 40%, doubled the context length of the model, and adopted grouped-query attention (Ainslie et al., 2023). We are releasing variants of Llama 2 with 7B, 13B, and 70B parameters. We have also trained 34B variants, which we report on in this paper but are not releasing.
"""

# 2. 构建 Prompt 阶段 (Augmentation)
# 将检索到的信息填充到提示词模板中
prompt = build_prompt(prompt_template,  # 提示词模板
                      info=search_results,  # 已知信息（检索结果）
                      query=user_query)  # 用户问题

print("===================== 构建的Prompt =====================")
print(prompt)

# 3. 生成阶段 (Generation)
# 调用DeepSeek模型基于增强后的提示词生成回答
response = get_completion(prompt)

# 输出最终回复
print("\n===================== DeepSeek回复 =====================")
print(response)

# 可选：打印调试信息
print("\n===================== 调试信息 =====================")
print(f"用户查询: {user_query}")
print(f"检索结果长度: {len(search_results)} 字符")
print(f"最终Prompt长度: {len(prompt)} 字符")

```

示例二

```python
import json
import os
import requests  # 导入requests库用于HTTP请求
from dotenv import load_dotenv, find_dotenv
from global_var import *

# DeepSeek API配置
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
DEEPSEEK_API_KEY = ds_key


def get_statistics_file(file_path, lines_num):
```

```python
    """
    逐批读取文件内容，每次返回指定行数的内容

    参数:
        file_path: 文件路径
        lines_num: 每次读取的行数

    返回:
        生成器，每次生成包含指定行数字符串
    """
    ret = ""  # 初始化返回字符串
    num = 0  # 行数计数器

    try:
        with open(file_path, 'r', encoding="utf-8") as f:  # 以只读模式打开文件，使用UTF-8编码
            for line in f.readlines():  # 逐行读取文件内容
                ret += line  # 将每行内容拼接到返回字符串

                if "\n" in line:  # 如果行中包含换行符（表示一行结束）
                    num += 1  # 行数计数器加1

                if num >= lines_num:  # 如果已经读取了指定行数
                    yield ret  # 返回当前累积的内容
                    ret = ""  # 重置返回字符串
                    num = 0  # 重置行数计数器

        # 返回剩余的内容（如果行数不足指定数量）
        if ret:
            yield ret

    except FileNotFoundError:
        print(f"错误：文件 {file_path} 未找到")
        yield ""
    except Exception as e:
        print(f"读取文件时发生错误: {e}")
        yield ""


def save_to_file(content, file_path):
    """
    将内容保存到文件中（追加模式）

    参数:
        content: 要保存的内容
        file_path: 文件路径
    """
    try:
        with open(file_path, 'a', encoding='utf-8') as f:  # 以追加模式打开文件
            lines = content.split('\n')  # 按换行符拆分内容
            for line in lines:
                if line.strip():  # 只保存非空行
                    f.write(line + '\n')  # 写入行并添加换行符
        print(f"内容已成功保存到文件: {file_path}")
    except Exception as e:
        print(f"保存文件时发生错误: {e}")

```

```python

def get_completion(prompt, model="deepseek-chat"):
    """
    调用DeepSeek API获取对话补全结果

    参数:
        prompt: 用户输入的提示词文本
        model: 使用的DeepSeek模型名称，默认为"deepseek-chat"

    返回:
        DeepSeek API的回复内容
    """
    # 构建消息列表，包含用户角色和内容
    messages = [{"role": "user", "content": prompt}]

    # 设置API请求头
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",  # Bearer令牌认证
        "Content-Type": "application/json"  # 请求内容类型为JSON
    }

    # 构建API请求负载
    payload = {
        "model": model,  # 指定使用的模型
        "messages": messages,  # 对话内容
        "temperature": 0,  # 模型输出的随机性，0表示随机性最小（最确定性）
    }

    try:
        # 发送POST请求到DeepSeek API
        response = requests.post(DEEPSEEK_API_URL, headers=headers, json=payload)
        response.raise_for_status()  # 如果请求失败则抛出异常

        # 解析JSON响应
        response_data = response.json()
        print("DeepSeek API响应状态: 成功")

        # 返回第一条选择的消息内容
        return response_data["choices"][0]["message"]["content"]

    except requests.exceptions.RequestException as e:
        # 处理网络请求错误
        print(f"API请求错误: {e}")
        return "抱歉，API请求失败，请稍后重试。"
    except KeyError as e:
        # 处理响应格式错误
        print(f"响应格式错误: {e}")
        return "抱歉，API响应格式异常。"


def build_prompt(prompt_template, **kwargs):
    """
    构建动态提示词，将模板中的变量替换为实际值

    参数:
        prompt_template: 提示词模板字符串，包含__变量名__格式的占位符
        **kwargs: 关键字参数，用于替换模板中的变量
```

```python

    返回:
        替换后的完整提示词字符串
    """
    prompt = prompt_template  # 初始化提示词为模板

    # 遍历所有关键字参数
    for k, v in kwargs.items():
        if isinstance(v, str):  # 如果值是字符串类型
            val = v
        elif isinstance(v, list) and all(isinstance(elem, str) for elem in v):  # 如果值是字符串列表
            val = '\n'.join(v)  # 用换行符连接列表中的所有字符串
        else:  # 其他数据类型
            val = str(v)  # 转换为字符串

        # 替换模板中的占位符（格式：__变量名__）
        placeholder = f"__{k.upper()}__"  # 构造占位符，如__INFO__
        prompt = prompt.replace(placeholder, val)  # 执行替换

    return prompt


# RAG提示词模板
# 这个模板指导模型如何基于检索到的信息回答问题
prompt_template = """
你是一个问答机器人。
你的任务是根据下述给定的已知信息回答用户问题。
确保你的回复完全依据下述已知信息。不要编造答案。
如果下述已知信息不足以回答用户的问题，请直接回复"我无法回答您的问题"。

已知信息:
__INFO__

用户问:
__QUERY__

请用中文回答用户问题。
"""

# 用户查询问题 - 关于一季度GDP的统计问题
user_query = """
问题一：第一季度，房地产开发投资金额是多少，增长或减少百分比为多少？
问题二：截止第一节度末，商品房待售面积是多少，增长或减少百分比是多少？
问题三：根据以上数据，分析可能是什么原因导致发生了这些变化？可以从行业、政策、公司经营角度来阐述
"""

print("开始RAG处理流程...")
print(f"用户查询: {user_query.strip()}")

# 1. 检索阶段 (Retrieval)
# 分批读取文件内容，每次读取2行，避免一次性加载大文件
for search_results in get_statistics_file("test.txt", 2):
    if not search_results.strip():  # 如果检索结果为空
        print("检索结果为空，跳过该批次")
        continue

```

```python
    print(f"\n处理批次内容 (长度: {len(search_results)} 字符):")
    print("=" * 50)

    # 2. 构建 Prompt 阶段 (Augmentation)
    # 将检索到的信息填充到提示词模板中
    prompt = build_prompt(prompt_template,
                          info=search_results,  # 已知信息（检索结果）
                          query=user_query)  # 用户问题

    print("=== 构建的Prompt ===")
    print(prompt)
    print("=" * 50)

    # 3. 生成阶段 (Generation)
    # 调用DeepSeek模型基于增强后的提示词生成回答
    print("调用DeepSeek API生成回答...")
    response = get_completion(prompt)

    print("\n=== DeepSeek回复 ===")
    print(response)
    print("=" * 50)

    # 检查回复是否包含"无法回答"，如果包含则继续检索下一批内容
    if "无法回答" in response:
        print("当前批次信息不足，继续检索下一批...")
        continue
    else:
        # 如果成功回答问题，保存结果到文件并退出循环
        save_to_file(response, "output.txt")
        print("回答已写入文件，处理完成。")
        break  # 退出循环，不再处理后续批次

print("\nRAG处理流程结束。")
```

5.4 Embedding

5.4.1 什么是Embedding

Embedding（嵌入）是大语言模型中的一项核心技术，它**将离散的文本符号转换为连续的数值向量**，这个向量中包含了词语、句子或文档的语义信息（形式为抽象高维数值表示）。

5.4.2 为什么需要Embedding

1）解决计算机理解文本语义问题

计算机天生只理解数字，无法直接处理人类语言。Embedding将人类文字（或文本）转换为计算机可以理解、容易计算的数值，建立起了人类语言与机器理解之间的桥梁。

2）语义相似度计算

传统的文本匹配基于关键词重合度，这种浅层次方法用于深层次语义理解效果不好。而Embedding能够表达深层次的语义信息，即使它们没有共同词汇，例如“我有点饿”和“想找个地方吃点”在embedding方式看来语义是一致的。

3）提升模型性能

Embedding让大模型能够更好地理解语言上下文、处理一词多义、识别情感倾向。例如，"苹果"这个词在"吃苹果"和"苹果手机"中的Embedding表示是不同的，模型能够根据上下文区分其含义。

4）支持下游任务

机器翻译、文本分类、情感分析、问答系统等几乎所有NLP任务都依赖于高质量的Embedding表示。

5.4.3 如何获取Embedding向量

Embedding主要通过深度学习模型，通过海量文本训练，在训练过程中，模型观察词汇在上下文中的出现模式，调整向量表示，使得语义相似的词汇在向量空间中位置接近。可以使用传统NLP小模型训练，也可以使用大模型训练，也可以使用LLM服务商提供的API进行生成。

5.4.4 Embedding使用示例

由于DeepSeek没有提供Embedding API，所以本小节案例使用火山引擎模型演示（火山引擎配置方法见《大模型开发环境安装手册.md》）

示例：利用大模型API生成文本Embedding表示，并计算句子之间的相似度，根据相似度检索出最匹配的句子

```python
import os
import numpy as np
from volcenginesdkarkruntime import Ark

# 初始化火山引擎客户端，使用API密钥进行认证
api_key = "2e*****************************b" # 这里改成自己的api_key
client = Ark(api_key=api_key)

# 知识库
knowledge_base = [
    "机器学习是人工智能的一部分",
    "自然语言处理处理文本数据",
    "计算机视觉识别图像内容",
    "数据仓库用来存储海量数据",
    "数据透视表用来分析数据",
    "计算机网络用来连接计算机设备"
]

# 查询句子
query = "人工智能技术的作用"
TOP_K = 3  # 可以调整这个值来控制返回的数量

# 获取所有嵌入向量：将文本转换为数值向量表示
all_texts = [query] + knowledge_base  # 组合查询和知识库文本
resp = client.embeddings.create(
    model="doubao-embedding-text-240715",  # 使用豆包文本嵌入模型
    input=all_texts,  # 输入文本列表
    encoding_format="float",  # 指定返回向量为浮点数格式
)

# 从API响应中提取嵌入向量
embeddings = [item.embedding for item in resp.data]
query_embedding = embeddings[0]  # 第一个向量是查询文本的嵌入表示
kb_embeddings = embeddings[1:]   # 剩余向量是知识库句子的嵌入表示

# 计算余弦相似度函数：衡量两个向量方向的相似性
def cosine_sim(a, b):
    a, b = np.array(a), np.array(b)  # 转换为numpy数组以便进行数学运算
    dot_product = np.dot(a, b)       # 计算向量点积（内积）
    norm_a = np.linalg.norm(a)       # 计算向量a的模长（欧几里得范数）
```

```python
    norm_b = np.linalg.norm(b)        # 计算向量b的模长
    if norm_a == 0 or norm_b == 0:    # 处理零向量情况，避免除零错误
        return 0.0

    return dot_product / (norm_a * norm_b)  # 返回余弦相似度值，范围[0,1]

# 计算查询与知识库中所有句子的相似度
results = []
for i, emb in enumerate(kb_embeddings):
    similarity = cosine_sim(query_embedding, emb)  # 计算当前句子与查询的相似度
    results.append((knowledge_base[i], similarity, i))  # 存储句子文本、相似度得分和原始索引

# 按相似度降序排序，最相似的排在前面
results.sort(key=lambda x: x[1], reverse=True)

# 调试输出：显示所有句子的相似度排序结果
print("根据相似度排序结果:")
for sentence, score, original_idx in results:
    print(f"{score:.4f}: {sentence}")

print("-" * 50)

# 只取前K个结果，实现Top-K相似度检索
top_results = results[:TOP_K]

# 输出最终结果
print(f"查询: '{query}'")
print(f"\n前{TOP_K}个最相关的句子:")
print("-" * 50)

# 格式化输出前K个最相似的结果
for i, (sentence, score, original_idx) in enumerate(top_results, 1):
    print(f"{i}. 相似度: {score:.4f}")
    print(f"   句子: {sentence}")
    print(f"   原始位置: 知识库第{original_idx + 1}条")  # 显示句子在原始知识库中的位置
    print()

# 显示统计信息：提供整体相似度分布的概览
print("=" * 50)
print(f"总共 {len(results)} 个句子，显示了相似度最高的前 {TOP_K} 个")
if len(results) > TOP_K:
    print(f"最低相似度: {results[-1][1]:.4f}")  # 所有结果中的最低相似度
    print(f"平均相似度: {sum(x[1] for x in results) / len(results):.4f}")  # 平均相似度
```

6. 向量数据库

6.1 什么是向量数据库

向量数据库是一种专门设计用于存储、索引和检索向量数据（嵌入向量）的数据库系统。与传统的关系型数据库、非关系型数据库不同，向量数据库的核心能力在于**高效处理高维向量数据**，并能够快速找到与查询向量最相似的数据点。

从技术角度来看，向量数据库通过将非结构化数据（如文本、图像、音频）转换为高维向量表示，然后使用**近似最近邻搜索**（Approximate Nearest Neighbor, ANN）算法来加速相似性检索。这种设计使得向量数据库能够在大规模数据集上实现毫秒级的相似性搜索，这是传统数据库无法实现的。

向量数据库的核心组件包括：**向量索引结构**（如HNSW、IVF、PQ等）、**相似度度量算法**（如余弦相似度、欧氏距离、内积等）以及**分布式存储架构**。这些技术共同构成了向量数据库处理海量向量数据的能力基础。

6.2 向量数据库的实现原理

数据预处理阶段，向量数据库首先将原始数据通过嵌入模型转换为高维向量。这个过程可能使用预训练模型如BERT、ResNet，或领域特定的嵌入模型。

索引构建过程中，向量数据库使用专门的算法结构组织向量数据。常见的索引算法包括：HNSW（分层可导航小世界）图结构、IVF（倒排文件）聚类索引、PQ（乘积量化）压缩技术等。这些索引技术在精度和效率之间提供不同的权衡选择。

查询处理机制采用近似最近邻搜索算法。当用户输入查询时，系统先将查询转换为向量，然后在索引结构中进行快速相似性搜索。ANN算法通过牺牲少量精度来换取搜索速度的数量级提升。

相似度计算使用多种度量标准，包括余弦相似度（衡量向量方向相似性）、欧氏距离（衡量绝对距离）、内积相似度等。不同场景适合不同的相似度度量方式。

分布式架构设计使向量数据库能够处理海量数据。通过分片、复制和负载均衡技术，向量数据库可以水平扩展，支持PB级向量数据的存储和检索。

实时更新能力是现代向量数据库的重要特性。支持增量和实时索引更新，确保新数据能够及时被检索到，满足动态变化的应用需求。

向量数据库作为AI基础设施的关键组件，正在重塑数据存储和检索的方式。随着多模态AI的发展，向量数据库将进一步支持文本、图像、音频等多种类型数据的统一处理和检索，为人工智能应用提供更强大的数据支撑能力。

6.3 向量数据库的应用

1）文本搜索

向量数据库实现了真正的"意图理解"。用户可以用自然语言描述需求，系统通过向量相似度找到语义相关的结果，而不依赖关键词匹配。

2）推荐系统

推荐系统广泛使用向量数据库，如电商平台使用商品描述的向量表示来推荐相似商品；内容平台通过用户行为数据的向量化实现个性化推荐；社交网络利用向量检索发现相似用户或内容。

3）大语言模型RAG

在LLM应用中，通过RAG（检索增强生成）架构，向量数据库为LLM提供实时知识检索能力，使模型能够基于最新信息生成回答，减少幻觉现象。

4）图像和视频检索

在图像、视频检索领域，向量数据库处理视觉内容的嵌入向量，实现人脸比对、以图搜图、内容推荐、侵权检测等功能。

5）生物信息学和化学领域

向量数据库用于分子结构相似性搜索、基因序列比对、药物发现等专业应用。

6.4 业界主流向量数据库

数据库	开源/商业	License	GitHub 仓库	部署方式	索引类型	GPU 加速	元数据过滤	客户端语言	典型用例
Pinecone	商业（托管）	Proprietary	pinecone-io	云端 SaaS	HNSW、IVF、Flat	是（服务层面自动优化）	是	Python, JavaScript, Go, Java	企业级相似性搜索、推荐
Weaviate	开源+商业托管	BSD-3-Clause	weaviate/weaviate	本地 / 私有云 / WCS	HNSW、Annoy、Hybrid	否	是	GraphQL, REST; Python, Go, JavaScript	RAG、知识库
Milvus	开源	Apache-2.0	milvus-io/milvus	本地 / Kubernetes	HNSW、IVF、PQ、DiskANN	是（显式 GPU 支持）	是	Python, Go, Java, Node.js	大规模检索、机器学习
Qdrant	开源+云托管	Apache-2.0	qdrant/qdrant	本地 / 云服务	HNSW、Flat	否	是	Python, JavaScript, Go, Rust	语义匹配、载荷过滤
Chroma	开源	Apache-2.0	chroma-core/chroma	本地	HNSW、Flat	否	是	Python	小型 RAG、原型开发
Redis Stack	开源+商业支持	Source Available	RediSearch/RediSearch	本地 / 云（Redis Labs）	HNSW、FLAT	否	是	多语言	实时计算、低延迟检索
Vespa	开源	Apache-2.0	vespa-engine/vespa	本地 / 云	HNSW、Flat	否	是	Java, Python, REST	高并发在线推理、搜索
Elasticsearch	开源+商用	SSPL / Elastic License	elastic/elasticsearch	本地 / 云（Elastic）	Approx. kNN（HNSW、FLAT）	否	是	多语言	搜索+向量检索混合场景
FAISS	开源	MIT	facebookresearch/faiss	库（嵌入应用）	Flat、IVF-PQ、HNSW、DiskANN	是	否	C++, Python	定制化检索系统构建

1）Pinecone

Pinecone 是一款全托管的商业向量数据库服务，专注于企业级相似度检索与推荐系统，提供 REST 和 gRPC 接口、自动扩缩容、高可用 SLA 及 SOC 2/GDPR/ISO 27001/HIPAA 合规保障。

API 支持：向量 CRUD、批量插入、命名空间管理、过滤查询等。
开源工具：官方 SDK 开源于 GitHub 组织 https://github.com/pinecone-io

2）Weaviate

Weaviate 是一个 AI 原生的开源向量数据库，内置 GraphQL 接口和模块化插件（RAG、嵌入器、向量分类等），支持在单次查询中结合向量相似度与结构化过滤，既可 Docker/Kubernetes 本地部署，也可使用 Weaviate Cloud Services（WCS）。

开源地址：https://github.com/weaviate/weaviate

3）Milvus

Milvus 是一款云原生、高性能的开源向量数据库，支持 HNSW、IVF、PQ、DiskANN 等多种索引和 GPU 加速，能够在亿级向量规模下实现低延迟高吞吐。

生态集成：提供 Python、Go、Java、Node.js 客户端，并与 Haystack、LangChain 等框架无缝对接。
开源地址：https://github.com/milvus-io/milvus

4）Qdrant
Qdrant（取自 Quadrant）是一款用 Rust 编写的开源向量相似度搜索引擎，支持 HTTP/gRPC 接口，能够将向量与任意元数据（payload）联合过滤，实现灵活的语义匹配与分面检索，具备水平扩展与高可用特性。

开源地址：https://github.com/qdrant/qdrant

5）Chroma
Chroma 是一个开源的 AI 应用数据库，集成文档管理、向量存储、全文检索与元数据过滤，紧密结合 Python 生态（如 LangChain），便于快速原型开发与小规模部署。

开源地址：https://github.com/chroma-core/chroma

6）Redis Stack (Vector Search)
Redis Stack 通过 RediSearch 模块在原生 Redis 中引入 VECTOR 数据类型，支持 HNSW 与 FLAT 索引，实现亚毫秒级检索延迟与 99.999% 高可用性，适合对实时性要求极高的场景。

开源地址：https://github.com/RediSearch/RediSearch

7）Vespa
Vespa 是一个开源的分布式应用引擎，既能做文本搜索也可作为向量数据库使用，内置在线机器学习模型推理与实时特征计算，支持自动分片、负载均衡，适合高并发在线推理与搜索场景。

开源地址：https://github.com/vespa-engine/vespa

8）Elasticsearch
Elasticsearch 自 7.3 起通过 dense_vector 字段与 k-NN 插件支持向量相似度检索，结合全文搜索、聚合与脚本功能，能在同一平台上实现混合搜索与向量检索场景。

开源地址：https://github.com/elastic/elasticsearch

9）FAISS
FAISS 是 Meta（Facebook）开源的向量相似度搜索库，提供 Flat、IVF-PQ、HNSW、DiskANN 等多种索引算法和 GPU 加速，常用作自定义检索系统的底层组件，与 NumPy、PyTorch、TensorFlow 深度集成。

开源地址：https://github.com/facebookresearch/faiss

6.5 ChromaDB

6.5.1 简介

ChromaDB 是一个开源的**嵌入式向量数据库**，专为人工智能应用，特别是大型语言模型（LLM）应用而设计。其核心使命是简化AI应用开发中“知识”的存储与检索过程。

在传统的AI交互中，模型仅依赖于其内部训练好的参数来生成回答，这常常导致回答缺乏针对性、时效性或私有知识。ChromaDB通过引入“**检索增强生成（RAG）**”模式解决了这一痛点。开发者可以将外部知识（如公司文档、产品手册、最新资讯等）转换为**向量（Embeddings）** 并存入ChromaDB。当用户提问时，系统会先在ChromaDB中快速检索出最相关的知识片段，再将其作为上下文提供给LLM，从而生成更精准、更可靠的答案。

ChromaDB的设计目标是为开发者提供一种简单、高效的工具，帮助他们将现实世界中的知识、事实和技能整合到大模型中。其设计理念包括：

- 简单性与开发效率：ChromaDB的API设计简洁，开发者可以快速上手并集成到现有应用中。
- 搜索与分析并重：除了支持高效的相似度搜索，ChromaDB还提供了对搜索结果的分析功能，帮助开发者更好地理解数据。
- 高性能：ChromaDB在保证功能丰富的同时，也追求极致的性能表

6.5.2 特点

1）支持多种存储后端

ChromaDB支持多种底层存储选项，如DuckDB（适用于独立应用）和ClickHouse（适用于大规模扩展）。

2）多语言支持

ChromaDB提供了Python和JavaScript/TypeScript的SDK，方便开发者快速集成。

3）简单易用

ChromaDB的设计理念是“简单至上”，旨在提升开发者的效率。

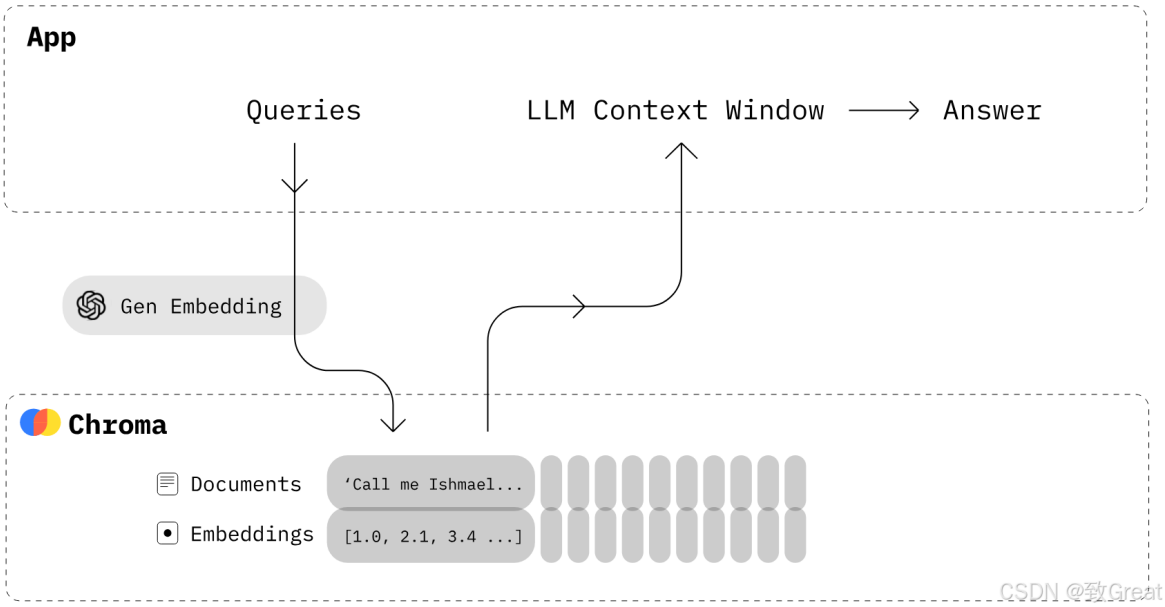
4）高性能

ChromaDB不仅支持快速的相似度搜索，还提供了对搜索结果的分析功能。

6.5.3 工作流程

ChromaDB的工作流程可以分为以下几个步骤：

- **创建集合（Collection）**：集合类似于关系数据库中的表，用于存储文档及其嵌入。默认情况下，Chroma使用`all-MiniLM-L6-v2`模型将文本转换为嵌入，但开发者可以根据需求选择其他嵌入模型。
- **添加文档**：将文本文档及其元数据添加到集合中。Chroma会自动将文本转换为嵌入并存储。
- **查询与搜索**：通过文本或嵌入查询集合，Chroma会返回与查询内容相似的文档。开发者还可以根据元数据对结果进行过滤。



6.5.4 安装方法

Chromadb安装使用如下指令（可以根据自己的实际情况，更换其它安装源）：

```
pip install chromadb==0.3.29 --index-url
https://pypi.tuna.tsinghua.edu.cn/simple/
```

6.5.5 使用示例

示例一：将文本转换为向量，插入ChromaDB并进行检索

```python
# ChromaDB存储、查询数据，火山引擎实现Embedding
import os
import numpy as np
from volcenginesdkarkruntime import Ark
import chromadb
from chromadb.config import Settings

# 初始化火山引擎客户端，使用API密钥进行认证
# 这里需要替换为实际的API密钥
api_key = "2e7************************7b" # 这里换成你自己的key
client = Ark(api_key=api_key)

# 知识库句子集合，包含多个技术相关的文本片段
# 这些文本将作为向量数据库中的参考数据，用于相似度匹配
knowledge_base = [
    "机器学习是人工智能的一部分",
    "自然语言处理处理文本数据",
    "计算机视觉识别图像内容",
    "数据仓库用来存储海量数据",
    "数据透视表用来分析数据",
    "计算机网络用来连接计算机设备"
]

# 用户输入的查询文本，系统将寻找与这个查询最相似的知识库句子
query = "人工智能技术的作用"

# 设置返回最相似结果的数量，可以调整这个值来控制返回结果的多少
TOP_K = 3


# 自定义嵌入函数类，用于将文本转换为向量表示
class VolcEngineEmbeddingFunction:
    def __init__(self, api_key):
        # 初始化API密钥和客户端
        self.api_key = api_key
        self.client = Ark(api_key=api_key)
        # 指定使用的嵌入模型名称
        self.model = "doubao-embedding-text-240715"

    def __call__(self, input_texts):
        """将输入的文本列表转换为向量列表"""
        try:
            print("正在将文本转换为向量...")
            # 调用火山引擎的嵌入API，将文本转换为向量
            resp = self.client.embeddings.create(
                model=self.model,  # 指定模型
                input=input_texts,  # 输入文本列表
                encoding_format="float",  # 返回浮点数格式的向量
            )

            # 检查API响应是否包含有效数据
            if resp.data and len(resp.data) > 0:
                # 从响应中提取所有嵌入向量
```

```
                embeddings_list = [item.embedding for item in resp.data]
                return embeddings_list
            else:
                # 如果API返回空数据，抛出异常
                raise ValueError("火山引擎API返回空数据")

        except Exception as e:
            # 处理API调用过程中可能出现的错误
            print(f"火山引擎SDK调用出错: {e}")
            raise


# 初始化ChromaDB向量数据库
def initialize_chromadb():
    """设置并返回ChromaDB集合对象"""
    # 创建ChromaDB客户端，配置持久化存储路径和数据库实现方式
    chroma_client = chromadb.Client(
        Settings(
            persist_directory="./chroma_volc_knowledge_db",  # 数据保存目录
            chroma_db_impl="duckdb+parquet"  # 使用DuckDB和Parquet格式
        )
    )

    # 创建嵌入函数实例，用于文本到向量的转换
    embedding_function = VolcEngineEmbeddingFunction(api_key=api_key)

    # 创建或获取一个名为"knowledge_base_collection"的集合
    collection = chroma_client.get_or_create_collection(
        name="knowledge_base_collection", # 集合名称
        embedding_function=embedding_function # 嵌入函数
    )

    return collection


# 将知识库数据存储到ChromaDB中
def store_knowledge_to_chromadb(collection, knowledge_base):
    """将知识库文本添加到向量数据库中"""
    # 为每个知识库条目生成唯一的标识符
    ids = [f"kb_{i}" for i in range(len(knowledge_base))]

    # 为每个条目添加元数据，记录类型和在原始列表中的位置
    metadatas = [{"type": "knowledge", "index": i} for i in range(len(knowledge_base))]

    print("正在将知识库存储到ChromaDB...")

    # 将文档、元数据和ID一起添加到集合中
    collection.add(
        documents=knowledge_base,  # 文本内容
        metadatas=metadatas,  # 附加信息
        ids=ids  # 唯一标识符
    )

    print(f"成功存储 {len(knowledge_base)} 个知识条目")
    return collection


```

```python
# 在ChromaDB中执行查询
def query_chromadb(collection, query_text, top_k=TOP_K):
    """在向量数据库中搜索与查询文本最相似的条目"""
    print(f"\n正在查询: '{query_text}'")

    # 执行相似度查询，返回指定数量的最相似结果
    results = collection.query(
        query_texts=[query_text],  # 查询文本
        n_results=top_k,  # 返回结果数量
        include=["documents", "metadatas", "distances"]  # 包含的信息类型
    )

    return results


# 显示查询结果
def display_results(query, results):
    """以友好格式展示查询结果"""
    print(f"查询: '{query}'")
    print(f"\n前{len(results['documents'][0])}个最相关的句子:")
    print("-" * 50)

    # 遍历所有结果并显示详细信息
    docs = results['documents'][0]# 文档内容
    metadatas = results['metadatas'][0]# 元数据
    distances = results['distances'][0]# 距离值
    for i, (doc, meta, dist) in enumerate(zip(docs, metadatas, distances)):
        # 将距离转换为相似度（距离越小，相似度越高）
        similarity = 1 - dist
        print(f"{i + 1}. 相似度: {similarity:.4f}")
        print(f"   句子: {doc}")
        print(f"   原始位置: 知识库第{meta['index'] + 1}条")
        print()


# 主函数，程序的入口点
def main():
    """执行主要的程序逻辑"""
    try:
        # 初始化ChromaDB数据库连接
        collection = initialize_chromadb()

        # 检查集合中是否已有数据
        if collection.count() == 0:
            # 如果集合为空，插入知识库数据
            collection = store_knowledge_to_chromadb(collection, knowledge_base)
        else:
            # 如果已有数据，显示当前数据量并跳过插入
            print("ChromaDB中已有数据，跳过插入步骤")
            print(f"当前集合中有 {collection.count()} 个文档")

        # 执行主要查询
        results = query_chromadb(collection, query, TOP_K)

        # 显示查询结果
        display_results(query, results)

```

```
        # 显示总体统计信息
        print("=" * 50)
        print(f"总共 {collection.count()} 个知识条目，显示了相似度最高的前 {TOP_K} 个")

        # 演示额外的查询示例
        print("\n" + "=" * 50)
        print("额外查询示例:")
        print("=" * 50)

        # 定义额外的查询文本
        additional_queries = ["文本分析", "图像识别", "数据存储"]
        for additional_query in additional_queries:
            # 对每个额外查询执行搜索
            additional_results = query_chromadb(collection, additional_query, 2)
            # 显示每个查询的结果
            display_results(additional_query, additional_results)

    except Exception as e:
        # 捕获并显示任何运行时错误
        print(f"程序执行出错: {e}")
        import traceback
        # 打印详细的错误堆栈信息，便于调试
        traceback.print_exc()


if __name__ == "__main__":
    main()
```

示例二：修改、删除操作。ChromaDB主要进行插入和检索，有时候需要对数据进行删除或修改。如下所示：

```
# ChromaDB文档管理操作（包含删除和修改）
import chromadb
from chromadb.config import Settings
from volcenginesdkarkruntime import Ark

api_key = "2e*****************************7b" # 这里改成你的火山API key


class VolcEngineEmbeddingFunction:
    """
    火山引擎嵌入函数类
    负责将文本转换为向量表示，用于ChromaDB的相似性搜索
    """

    def __init__(self, api_key):
        self.api_key = api_key  # 保存API密钥
        self.client = Ark(api_key=api_key)  # 创建火山引擎客户端实例
        self.model = "doubao-embedding-text-240715"  # 指定使用的嵌入模型

    def __call__(self, input_texts):
        """
        将输入的文本列表转换为向量列表

        参数:
```

```
        - input_texts: 字符串列表，需要转换为向量的文本

        返回:
        - List[List[float]]: 向量列表，每个文本对应一个向量

        """
        try:
            # 调用火山引擎嵌入API创建文本嵌入
            resp = self.client.embeddings.create(
                model=self.model,  # 指定模型名称
                input=input_texts,  # 输入文本列表
                encoding_format="float",  # 指定返回格式为浮点数
            )

            # 检查API响应是否包含有效数据
            if resp.data and len(resp.data) > 0:
                # 从响应中提取所有嵌入向量
                embeddings_list = [item.embedding for item in resp.data]
                return embeddings_list
            else:
                # 如果API返回空数据，抛出异常
                raise ValueError("火山引擎API返回空数据")

        except Exception as e:
            # 处理API调用过程中可能出现的错误
            print(f"火山引擎SDK调用出错: {e}")
            raise  # 重新抛出异常，让上层调用者处理


def initialize_chromadb():
    """
    初始化ChromaDB客户端并获取集合对象
    """
    # 创建ChromaDB客户端实例，配置持久化设置
    chroma_client = chromadb.Client(
        Settings(
            persist_directory="./chroma_volc_knowledge_db",  # 数据库文件存储路径
            chroma_db_impl="duckdb+parquet"  # 使用DuckDB引擎和Parquet格式存储
        )
    )

    # 初始化嵌入函数实例，用于文本到向量的转换
    embedding_function = VolcEngineEmbeddingFunction(api_key=api_key)

    # 获取或创建集合，并配置嵌入函数和相似度度量方式
    collection = chroma_client.get_or_create_collection(
        name="knowledge_base_collection",  # 集合名称
        embedding_function=embedding_function,  # 指定嵌入函数
        metadata={"hnsw:space": "cosine"}  # 使用余弦相似度作为距离度量
    )

    return collection, chroma_client  # 返回集合和客户端对象


def display_collection_info(collection):
    """
    显示集合的基本统计信息
```

```python
    """
    # 获取集合中的文档总数
    count = collection.count()
    # 打印集合信息和文档数量
    print(f"集合 '{collection.name}' 中共有 {count} 个文档")
    return count  # 返回文档数量


def display_all_documents(collection):
    """
    显示集合中的所有文档及其详细信息
    """
    # 打印分隔线和标题
    print("\n" + "=" * 60)
    print("当前集合中的所有文档:")
    print("=" * 60)

    # 获取集合中的所有文档数据，包括ID、文档内容、元数据
    results = collection.get()

    # 检查集合是否为空（没有文档ID）
    if not results['ids']:
        print("集合为空，没有文档")
        return  # 直接返回，不执行后续代码

    # 遍历所有文档并显示详细信息
    for i, (doc_id, doc, meta) in enumerate(zip(results['ids'], results['documents'], results['metadatas'])):
        # 显示文档序号和ID
        print(f"{i + 1}. ID: {doc_id}")
        # 显示文档内容，长文本截断前50个字符
        print(f"   内容: {doc[:50]}..." if len(doc) > 50 else f"   内容: {doc}")
        # 如果存在元数据，显示元数据信息
        if meta:
            print(f"   元数据: {meta}")
        print()  # 打印空行，分隔不同的文档


def update_document(collection):
    """
    更新特定文档的内容和元数据
    """
    print("\n" + "=" * 60)
    print("更新文档内容")
    print("=" * 60)

    # 先显示所有文档，方便用户选择要更新的文档
    display_all_documents(collection)

    # 获取用户输入的文档ID，并去除首尾空白字符
    document_id = input("请输入要更新的文档ID: ").strip()
    if not document_id:
        print("文档ID不能为空")
        return False  # 返回失败状态

    try:
        # 检查指定ID的文档是否存在
```

```python
        existing_doc = collection.get(ids=[document_id])
        if not existing_doc['ids']:
            print(f"文档ID '{document_id}' 不存在")
            return False  # 返回失败状态

        # 获取当前文档内容和元数据
        current_content = existing_doc['documents'][0]  # 当前文档内容
        # 安全地获取元数据，处理可能为空的情况
        current_metadata = existing_doc['metadatas'][0] if existing_doc['metadatas'] and len(
            existing_doc['metadatas']) > 0 else {}

        # 显示当前文档信息
        print(f"当前文档内容: {current_content}")
        if current_metadata:
            print(f"当前元数据: {current_metadata}")

        # 获取新的文档内容
        print("\n请输入新的文档内容（直接回车保持原内容不变）:")
        new_content = input("新内容: ").strip()
        if not new_content:
            new_content = current_content  # 如果用户输入为空，保持原内容

        # 复制当前元数据作为新元数据的基础
        new_metadata = current_metadata.copy() if current_metadata else {}

        # 显示更新信息让用户确认
        print(f"\n新内容: {new_content}")
        print(f"新元数据: {new_metadata}")
        # 请求用户确认更新操作
        confirm = input("确认更新这个文档吗？(y/n): ").lower()

        if confirm == 'y':
            # 采用先删除再添加的策略来更新文档
            # 这是因为直接使用update方法可能会导致嵌入函数相关问题
            collection.delete(ids=[document_id])  # 先删除旧文档
            collection.add(
                documents=[new_content],  # 新文档内容
                metadatas=[new_metadata],  # 新元数据
                ids=[document_id]  # 保持相同的文档ID
            )
            print(f"成功更新文档: ID '{document_id}'")
            return True  # 返回成功状态
        else:
            print("取消更新操作")
            return False  # 返回取消状态

    except Exception as e:
        # 捕获并处理更新过程中可能出现的异常
        print(f"更新文档时出错: {e}")
        import traceback
        traceback.print_exc()  # 打印详细的错误堆栈信息
        return False  # 返回失败状态


def delete_specific_document(collection):
    """
    根据文档ID删除单个特定文档
```

```python
    """
    print("\n" + "=" * 60)
    print("删除特定文档")
    print("=" * 60)

    # 先显示所有文档，方便用户选择要删除的文档
    display_all_documents(collection)

    # 获取用户输入的文档ID
    document_id = input("请输入要删除的文档ID: ").strip()
    if not document_id:
        print("文档ID不能为空")
        return False  # 返回失败状态

    try:
        # 检查文档是否存在
        results = collection.get(ids=[document_id])
        if not results['ids']:
            print(f"文档ID '{document_id}' 不存在")
            return False  # 返回失败状态

        # 显示找到的文档内容，让用户确认
        print(f"找到文档: {results['documents'][0]}")
        # 请求用户确认删除操作
        confirm = input("确认删除这个文档吗？(y/n): ").lower()

        if confirm == 'y':
            # 执行删除操作
            collection.delete(ids=[document_id])
            print(f"成功删除文档: ID '{document_id}'")
            return True  # 返回成功状态
        else:
            print("取消删除操作")
            return False  # 返回取消状态

    except Exception as e:
        # 处理删除过程中可能出现的异常
        print(f"删除文档时出错: {e}")
        return False  # 返回失败状态


def clear_entire_collection(collection):
    """
    清空整个集合中的所有文档
    """
    print("\n" + "=" * 60)
    print("清空整个集合")
    print("=" * 60)

    # 获取当前集合中的文档总数
    count = collection.count()
    if count == 0:
        print("集合已经是空的")
        return True  # 返回成功状态（无需操作）

    # 警告用户这是一个危险操作
    print(f"警告：这将删除集合中的所有 {count} 个文档！")
    # 要求输入特定的确认字符串以防止误操作
```

```python
    confirm = input("确认清空整个集合吗？(输入'DELETE'确认): ")

    # 检查用户确认
    if confirm == 'DELETE':
        # 获取所有文档ID
        all_docs = collection.get()
        # 执行批量删除操作
        collection.delete(ids=all_docs['ids'])
        print(f"成功清空集合，删除了 {count} 个文档")
        return True  # 返回成功状态
    else:
        print("取消清空操作")
        return False  # 返回取消状态


def main():
    """
    主函数 - ChromaDB文档管理工具
    """
    try:
        # 初始化ChromaDB连接，获取集合和客户端对象
        collection, chroma_client = initialize_chromadb()

        # 主循环，持续显示菜单直到用户选择退出
        while True:
            # 打印菜单界面
            print("\n" + "=" * 60)
            print("ChromaDB 文档管理工具")
            print("=" * 60)
            # 显示当前集合信息和文档数量
            print(f"当前集合: {collection.name}")
            print(f"文档数量: {collection.count()}")
            print("=" * 60)
            # 功能菜单选项
            print("1. 显示所有文档")
            print("2. 更新文档内容")
            print("3. 删除特定文档")
            print("4. 清空整个集合")
            print("5. 退出")
            print("=" * 60)

            # 获取用户输入的选择，去除首尾空白
            choice = input("请选择操作 (1-5): ").strip()

            # 根据用户选择执行相应的功能
            if choice == '1':
                display_all_documents(collection)  # 显示所有文档

            elif choice == '2':
                update_document(collection)  # 更新文档内容

            elif choice == '3':
                delete_specific_document(collection)  # 删除特定文档

            elif choice == '4':
                clear_entire_collection(collection)  # 清空整个集合

            elif choice == '5':
```

```
                print("退出程序")
                break  # 跳出循环，结束程序

            else:
                # 处理无效输入
                print("无效选择，请重新输入")

            # 暂停程序，等待用户按键继续
            input("\n按回车键继续...")

    except Exception as e:
        # 捕获并处理主程序中可能出现的任何异常
        print(f"程序执行出错: {e}")
        import traceback
        # 打印完整的错误堆栈跟踪，便于调试
        traceback.print_exc()


if __name__ == "__main__":
    main()
```

6. 6 ChromaDB+RAG综合案例

本案例读取PDF文档作为知识库，并将内容存入向量数据库，根据用户问题进行检索，返回检索的结果。

安装PDF操作库：

```
pip install pdfminer-six==20250506
```

代码：

```
from pdfminer.high_level import extract_pages
from pdfminer.layout import LTTextContainer
from dotenv import load_dotenv, find_dotenv
from volcenginesdkarkruntime import Ark  # 导入火山引擎SDK
import chromadb
from chromadb.config import Settings
import os
import time

# 初始化火山引擎客户端
api_key = "2e*****************************7b" # 这里改成你的火山API key


class VolcEngineEmbeddingFunction:
    """
    火山引擎嵌入函数类
    负责将文本转换为向量表示，用于ChromaDB的相似性搜索
    """

    def __init__(self, api_key):
        """
        初始化嵌入函数
        """
        self.api_key = api_key
```

```python
        self.client = Ark(api_key=api_key)
        self.model = "doubao-embedding-text-240715"  # 使用火山引擎的嵌入模型
        self.max_retries = 3  # 最大重试次数
        self.retry_delay = 5  # 重试延迟（秒）

    def __call__(self, input_texts):
        """
        将输入的文本列表转换为向量列表，带有重试机制
        """
        for attempt in range(self.max_retries):
            try:
                # 调用火山引擎的嵌入向量生成API
                resp = self.client.embeddings.create(
                    model=self.model,
                    input=input_texts,
                    encoding_format="float",
                )

                if resp.data and len(resp.data) > 0:
                    # 将结果转换为向量列表
                    embeddings_list = [item.embedding for item in resp.data]
                    return embeddings_list
                else:
                    raise ValueError("火山引擎API返回空数据")

            except Exception as e:
                if attempt < self.max_retries - 1:
                    print(f"火山引擎SDK调用出错 (尝试 {attempt + 1}/{self.max_retries}): {e}")
                    print(f"等待 {self.retry_delay} 秒后重试...")
                    time.sleep(self.retry_delay)
                    # 每次重试增加延迟时间
                    self.retry_delay *= 2
                else:
                    print(f"火山引擎SDK调用出错，已达最大重试次数: {e}")
                    # 返回空向量列表，让程序继续运行
                    return [[0.0] * 1024 for _ in input_texts]  # 返回默认向量


# 读取PDF文件内容
def extract_text_from_pdf(filename, page_numbers=None, min_line_length=1):
    '''从 PDF 文件中（按指定页码）提取文字'''
    paragraphs = []  # 存储段落
    buffer = ''  # 存储段落
    full_text = ''  # 存储全部文本

    # 提取全部文本
    for i, page_layout in enumerate(extract_pages(filename)):  # 遍历每一页
        # 如果指定了页码范围，跳过范围外的页
        if page_numbers is not None and i not in page_numbers:
            continue

        for element in page_layout:  # 遍历每页的每个元素
            if isinstance(element, LTTextContainer):  # 如果是文本元素
                full_text += element.get_text() + '\n'  # 将文本添加到full_text 中
```

```python

        print("读取完成:%d页" % (i + 1))
        if i >= 20:
            break

    # 按空行分隔，将文本重新组织成段落
    lines = full_text.split('\n')
    for text in lines:
        if len(text) >= min_line_length:  # 如果长度大于等于 min_line_length
            buffer += (' ' + text) if not text.endswith('-') else text.strip('-')
        elif buffer:  # 如果 buffer 不为空，则将 buffer 添加到 paragraphs 中
            paragraphs.append(buffer)
            buffer = ''

    if buffer:
        paragraphs.append(buffer)

    return paragraphs


# 读取PDF，并打印前几行
paragraphs = extract_text_from_pdf("llama2.pdf", min_line_length=10)
print("读取PDF完成")

# 创建火山引擎嵌入函数实例
volc_embedding_fn = VolcEngineEmbeddingFunction(api_key=api_key)


def get_embeddings(texts):
    """
    使用火山引擎获取文本嵌入向量
    """
    return volc_embedding_fn(texts)


# 为了演示方便，我们只取两页
paragraphs = extract_text_from_pdf(
    "llama2.pdf",  # 原始文件
    page_numbers=[2, 6],  # 读取指定的页码范围
    min_line_length=10  # 一行的最小长度
)


class MyVectorDBConnector:  # 自定义向量数据库连接器
    def __init__(self, collection_name, embedding_fn):
        # 创建 ChromaDB 客户端
        chroma_client = chromadb.Client(Settings(allow_reset=True))

        # 为了演示，实际不需要每次 reset()
        chroma_client.reset()

        # 获取一个集合
        self.collection = chroma_client.get_or_create_collection(name=collection_name)
        self.embedding_fn = embedding_fn

    def add_documents(self, documents):
```

```python
        '''向 collection 中添加文档与向量'''
        self.collection.add(
            embeddings=self.embedding_fn(documents),  # 将文档转换为向量
            documents=documents,  # 文档的原文
            ids=[f"id{i}" for i in range(len(documents))]  # 每个文档的 id
        )

    # 检索
    def search(self, query, top_n):
        '''检索向量数据库'''
        results = self.collection.query(  # 根据向量进行查询
            query_embeddings=self.embedding_fn([query]),
            n_results=top_n  # 指定返回跟query相似度最高的前N个结果
        )
        return results


# 创建一个向量数据库对象
vector_db = MyVectorDBConnector("demo", get_embeddings)
# 向向量数据库中添加文档
vector_db.add_documents(paragraphs)

user_query = "Llama 2有多少参数"
results = vector_db.search(user_query, 5)

for para in results['documents'][0]:
    print(para + "\n")
```

7. 高级检索

7.1 文档分片

在RAG（检索增强生成）系统中，文档分片是影响检索效果的关键环节。合理的分片策略能够确保语义完整性，提升向量检索的准确性。下面列举了常见的分片方式。

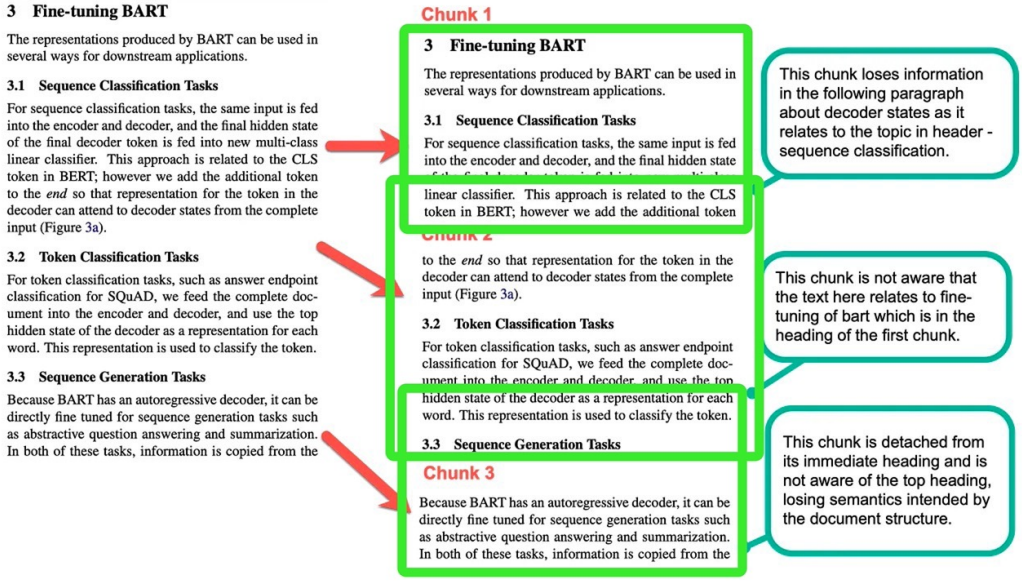
1）基于语义边界的分片

该方法是最为常见且有效的策略。这种方法利用自然语言的天然边界进行分割，包括：

- **句子分割**：使用NLTK、spaCy等工具识别句子边界，确保每个分片包含完整的语义单元
- **段落分割**：以空行或段落标记为界限，保持上下文的连贯性
- **语义完整性检测**：通过主题一致性或语义连贯性算法确定最佳分割点

2）重叠分片技术

重叠分片是提升检索召回率的重要方法，也是最常用的方法。通过在相邻分片间设置重叠区域（通常为50-100个字符），确保边界概念不会因分割而丢失。这种策略特别适合处理连续性强、概念跨段落的专业技术文档。



3）自适应分片策略

根据文档特性动态调整分片大小。技术文档可采用较小的分片（200-300字符）保证精确性，而文学性内容则适合较大的分片（500-800字符）保持语境完整。

4）混合分片方法

结合多种策略的优势：先按段落分割，再对过长段落进行句子级细分，最后应用重叠机制。这种分层 approach 在保持语义完整性的同时，确保了检索的粒度适宜。

在实际应用中，分片策略的选择需要综合考虑文档类型、查询特性和性能要求。良好的分片应该平衡检索精度与上下文完整性的关系，为后续的向量化处理和相似度检索奠定基础。以下是文档重叠分片的检索示例。

```python
import chromadb
from chromadb.config import Settings
from pdfminer.high_level import extract_pages
from pdfminer.layout import LTTextContainer
from volcenginesdkarkruntime import Ark  # 导入火山引擎SDK
import os
import time
from nltk.tokenize import sent_tokenize
import json

# 火山引擎API密钥
api_key = "2e*********************************7b"  # 嵌入模型API密钥


class VolcEngineEmbeddingFunction:
    """
    火山引擎嵌入函数类
    负责将文本转换为向量表示，用于ChromaDB的相似性搜索
    """

    def __init__(self, api_key):
        """
        初始化嵌入函数
        """
        self.api_key = api_key
```

```python
        self.client = Ark(api_key=api_key)
        self.model = "doubao-embedding-text-240715"  # 使用火山引擎的嵌入模型
        self.max_retries = 3  # 最大重试次数
        self.retry_delay = 5  # 重试延迟（秒）

    def __call__(self, input_texts):
        """
        将输入的文本列表转换为向量列表，带有重试机制
        """
        for attempt in range(self.max_retries):
            try:
                # 调用火山引擎的嵌入向量生成API
                resp = self.client.embeddings.create(
                    model=self.model,
                    input=input_texts,
                    encoding_format="float",
                )

                if resp.data and len(resp.data) > 0:
                    # 将结果转换为向量列表
                    embeddings_list = [item.embedding for item in resp.data]
                    return embeddings_list
                else:
                    raise ValueError("火山引擎API返回空数据")

            except Exception as e:
                if attempt < self.max_retries - 1:
                    print(f"火山引擎SDK调用出错 (尝试 {attempt + 1}/{self.max_retries}): {e}")
                    print(f"等待 {self.retry_delay} 秒后重试...")
                    time.sleep(self.retry_delay)
                    # 每次重试增加延迟时间
                    self.retry_delay *= 2
                else:
                    print(f"火山引擎SDK调用出错，已达最大重试次数: {e}")
                    # 返回空向量列表，让程序继续运行
                    return [[0.0] * 1024 for _ in input_texts]  # 返回默认向量


# 创建火山引擎嵌入函数实例
volc_embedding_fn = VolcEngineEmbeddingFunction(api_key=api_key)


def get_embeddings(texts):
    """
    使用火山引擎获取文本嵌入向量
    """
    return volc_embedding_fn(texts)


class MyVectorDBConnector:
    def __init__(self, collection_name, embedding_fn):
        chroma_client = chromadb.Client(Settings(allow_reset=True))

        # 为了演示，实际不需要每次 reset()
        chroma_client.reset()
```

```python

        # 创建一个 collection
        self.collection = chroma_client.get_or_create_collection(name=collection_name)
        self.embedding_fn = embedding_fn

    def add_documents(self, documents):
        '''向 collection 中添加文档与向量'''
        self.collection.add(
            embeddings=self.embedding_fn(documents),  # 每个文档的向量
            documents=documents,  # 文档的原文
            ids=[f"id{i}" for i in range(len(documents))]  # 每个文档的 id
        )

    def search(self, query, top_n):
        '''检索向量数据库'''
        results = self.collection.query(
            query_embeddings=self.embedding_fn([query]),
            n_results=top_n
        )
        return results


def extract_text_from_pdf(filename, page_numbers=None, min_line_length=1):
    '''从 PDF 文件中（按指定页码）提取文字'''
    paragraphs = []
    buffer = ''
    full_text = ''

    # 提取全部文本
    for i, page_layout in enumerate(extract_pages(filename)):
        # 如果指定了页码范围，跳过范围外的页
        if page_numbers is not None and i not in page_numbers:
            continue

        for element in page_layout:
            if isinstance(element, LTTextContainer):
                full_text += element.get_text() + '\n'

    # 按空行分隔，将文本重新组织成段落
    lines = full_text.split('\n')
    for text in lines:
        if len(text) >= min_line_length:  # 大于最小长度
            buffer += (' ' + text) if not text.endswith('-') else text.strip('-')
        elif buffer:
            paragraphs.append(buffer)
            buffer = ''

    if buffer:
        paragraphs.append(buffer)

    return paragraphs


class RAG_Bot:
    def __init__(self, vector_db, llm_api, n_results=2):
        self.vector_db = vector_db
```

```python
        self.llm_api = llm_api
        self.n_results = n_results

    def chat(self, user_query):
        # 1. 检索
        search_results = self.vector_db.search(user_query, self.n_results)

        # 2. 构建 Prompt
        prompt = build_prompt(prompt_template,  # 使用模板
                              info=search_results['documents'][0],  # 检索结果
                              query=user_query)  # 用户问题

        # 3. 调用 LLM
        response = self.llm_api(prompt)
        return response


def get_volcengine_completion(prompt,
                              model="deepseek-v3-250324",
                              max_tokens=2000,
                              temperature=0.1):
    """
    使用火山引擎LLM API获取对话完成
    """
    if not api_key:
        raise ValueError("火山引擎LLM API密钥未设置")

    try:
        # 初始化火山引擎LLM客户端 - 使用正确的base_url
        llm_client = Ark(
            base_url="https://ark.cn-beijing.volces.com/api/v3",
            api_key=api_key,
        )

        # 调用火山引擎的聊天补全API - 使用正确的参数格式
        completion = llm_client.chat.completions.create(
            model=model,  # 使用正确的模型名称
            # messages参数
            messages=[
                {"role": "user", "content": prompt}
            ],
            max_tokens=max_tokens,  # 最大生成长度
            temperature=temperature,  # 温度
            # 非流式响应
            stream=False
        )

        return completion.choices[0].message.content

    except Exception as e:
        print(f"火山引擎LLM API调用出错: {e}")
        return f"抱歉，火山引擎API调用出错: {str(e)}"


def split_text(paragraphs, chunk_size=300, overlap_size=100):
    '''按指定 chunk_size 和 overlap_size 交叠割文本'''
    # chunk_size: 片段大小
```

```python
    # overlap_size: 重叠区域大小
    sentences = [s.strip() for p in paragraphs for s in sent_tokenize(p)]
    chunks = []
    i = 0
    while i < len(sentences):
        chunk = sentences[i]
        overlap = ''
        prev_len = 0
        prev = i - 1
        # 向前计算重叠部分
        while prev >= 0 and len(sentences[prev]) + len(overlap) <= overlap_size:
            overlap = sentences[prev] + ' ' + overlap
            prev -= 1
        chunk = overlap + chunk
        next = i + 1
        # 向后计算当前chunk
        while next < len(sentences) and len(sentences[next]) + len(chunk) <= chunk_size:
            chunk = chunk + ' ' + sentences[next]
            next += 1
        chunks.append(chunk)
        i = next

    return chunks


def build_prompt(prompt_template, **kwargs):
    '''将 Prompt 模板赋值'''
    prompt = prompt_template
    for k, v in kwargs.items():
        if isinstance(v, str):
            val = v
        elif isinstance(v, list) and all(isinstance(elem, str) for elem in v):
            val = '\n'.join(v)
        else:
            val = str(v)
        prompt = prompt.replace(f"__{k.upper()}__", val)
    return prompt


prompt_template = """
你是一个问答机器人。
你的任务是根据下述给定的已知信息回答用户问题。
确保你的回复完全依据下述已知信息。不要编造答案。
如果下述已知信息不足以回答用户的问题，请直接回复"我无法回答您的问题"。

已知信息:
__INFO__

用户问:
__QUERY__

请用中文回答用户问题。
"""

# 为了演示方便，我们只取两页（第一章）
```

```python
paragraphs = extract_text_from_pdf("llama2.pdf",
                                   page_numbers=[2, 5],
                                   min_line_length=30)
# 切割文本，每个片段300字，重叠100字
chunks = split_text(paragraphs, 300, 100)
# 创建一个向量数据库对象
vector_db = MyVectorDBConnector("demo_text_split", get_embeddings)
# 向向量数据库中添加文档
vector_db.add_documents(chunks)  # 使用重叠切分的片段
# 创建一个RAG机器人，使用火山引擎LLM API
bot = RAG_Bot(vector_db, llm_api=get_volcengine_completion)

user_query = "llama 2可以商用吗？"
# user_query = "llama 2 chat有多少参数"

search_results = vector_db.search(user_query, 2)
print("检索到的相关文档:")
for i, doc in enumerate(search_results['documents'][0]):
    print(f"文档 {i + 1}: {doc[:100]}...\n")

# 将检索结果传递给LLM，整理成最终回复
response = bot.chat(user_query)
print("====回复====")
print(response)
```

7.2 检索后排序

检索后排序是指在初步向量检索的基础上，通过多维度特征分析对结果进行二次排序，提升最相关文档的排名位置。向量检索虽然能快速找到语义相似的文档，但无法充分考虑查询的具体关键词匹配、文本质量等因素。通过检索后排序，还可以将领域特定的排序规则（如时效性权重、权威性评分、用户偏好等）融入检索流程，使结果更符合具体应用场景的需求。检索后排序具有如下优点：

- **提升回答准确性**（核心优势）：通过将最相关的文档优先传递给LLM，显著提高生成答案的准确性和完整性。研究表明，优质检索结果能使LLM回答准确率提升20-30%。
- **改善用户体验**：返回的结果质量更高，减少无关或低质量信息的干扰，用户能更快获得满意答案。
- **资源优化**：在保持检索效果的前提下，可以减少传递给LLM的上下文数量，降低计算成本和token消耗。
- **灵活可控**：可以根据具体需求调整排序权重，比如在某些场景下提高时效性权重，在另一些场景下注重权威性。

检索后排序的注意事项：

- **性能平衡**：排序算法会增加计算开销，需要在效果和延迟之间找到平衡点。复杂的排序算法可能使检索时间增加50-100%。
- **过度工程风险**：避免设计过于复杂的排序规则，简单的规则组合往往比复杂模型更稳定可靠。
- **领域适应性**：不同领域的文本特征差异很大，需要针对特定领域调整排序策略。技术文档、新闻、学术论文等需要不同的排序偏好。
- **评估体系建立**：必须建立完善的评估指标来验证排序效果，包括相关性人工评估、答案质量对比测试等。
- **错误传播防范**：错误的排序可能放大检索阶段的错误，需要设置质量阈值和异常检测机制。

检索后排序是提升RAG系统效果的关键环节，通过精心设计的排序策略，可以在不显著增加成本的情况下大幅提升系统性能。

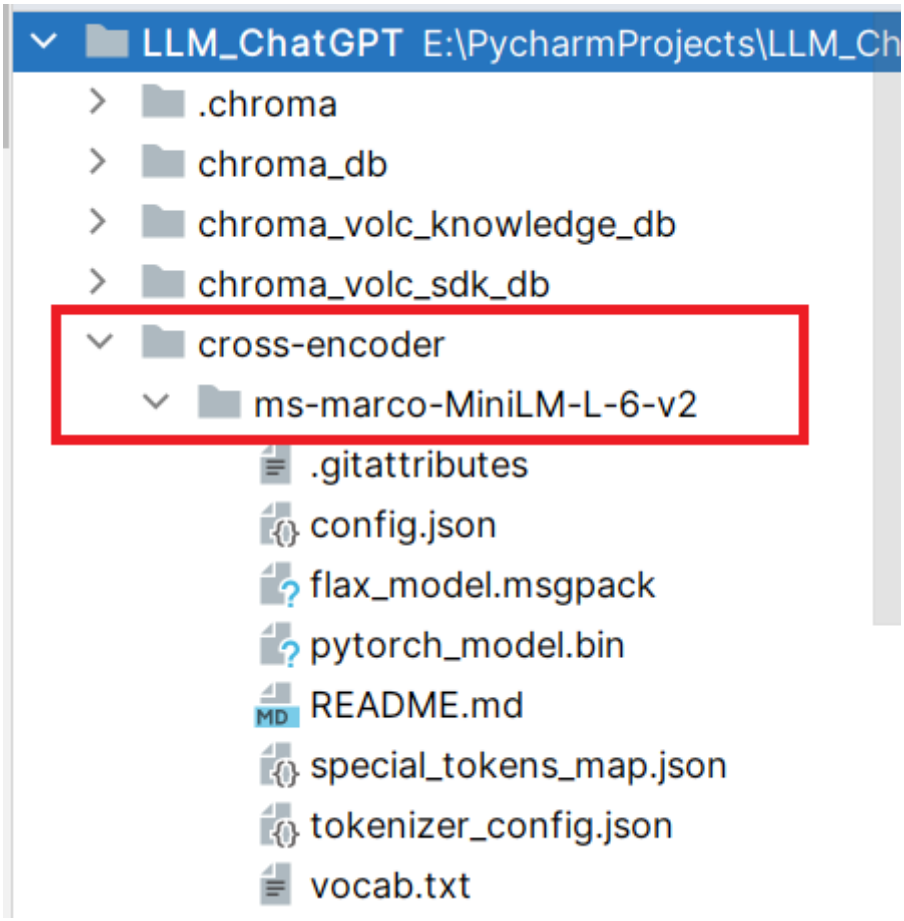
示例：检索后排序

- 第一步：安装sentence-transformers库

Sentence Transformers（又名 SBERT）是用于访问、使用和训练最先进的嵌入和重排序模型的首选 Python 模块。它可用于使用 Sentence Transformer 模型计算嵌入（快速入门），使用 Cross-Encoder（又名 reranker）模型计算相似度分数（快速入门），或使用 Sparse Encoder 模型生成稀疏嵌入（快速入门）。这开启了广泛的应用，包括语义搜索、语义文本相似度和转述挖掘。在 Hugging Face 上有超过 10,000 个预训练的 Sentence Transformers 模型可供立即使用，其中包括许多来自大规模文本嵌入基准 (MTEB) 排行榜的最先进模型。此外，使用 Sentence Transformers 训练或微调您自己的嵌入模型、重排序模型或稀疏编码器模型非常简单，使您能够为特定用例创建自定义模型。

```
pip install sentence-transformers==5.0.0 --index-url
https://pypi.tuna.tsinghua.edu.cn/simple/
```

- 第二步：下载ms-marco-MiniLM-L-6-v2.zip包，解压，在项目中创建cross-encoder目录，将解压后的目录拷贝到cross-encoder目录中，如下图所示：



- 第三步：将以下代码拷贝到上一个示例最后，执行

```
# 检索后排序
from sentence_transformers import CrossEncoder

# CrossEncoder：交叉编码器
# ms-marco-MiniLM-L-6-v2是一款针对MS Marco段落排序任务开发的跨编码器模型
# 该模型在信息检索领域表现卓越，能够高效编码和排序查询与文本段落
print("\n开始加载排序模型...")
model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2',  # 模型名称
                     max_length=512)  # 一个句子最多多少个词

print("加载模型完成，开始检索并排序.")

user_query = "how safe is llama 2"
```

```python
# predict：意思是预测，这里预测的结果是user_query和检索到的文字之间的匹配度
scores = model.predict([(user_query, doc)
                        for doc in search_results['documents'][0]])
# 按得分排序
sorted_list = sorted(zip(scores, search_results['documents'][0]),
                     key=lambda x: x[0],
                     reverse=True)  # 倒序排列
for score, doc in sorted_list:
    print(f"{score}\t{doc}\n")
```

7.3 改善RAG效果的策略

7.3.1 RAG应用的问题

在建立RAG应用过程中，可能会遇到以下一些问题：

- 问题比较抽象或者概念比较模糊，导致大模型没有准确理解使用者的问题。例如，使用者问“兰州拉面去哪吃？”，使用者本来想问附近有没有卖兰州拉面的店铺；假如知识库搜索“兰州”和“拉面”之后，结果排序靠前的语料是位于“兰州”的拉面馆地址，而大模型告诉用户要去买飞机票或者火车票，就是答非所问了。通常我们用改造问题，让使用者的问题更好理解的策略来回避这种情况。
- 知识库没有检索到问题的答案，这有可能是由于语料数据没有做好整理就存入知识库，或者是检索策略有问题，参数需要调整导致的。比如，在采用“K个最相似文档块”作为回答的知识这个策略中，如果K值比较小，那么最相似的K个文档块中可能并不包含能解答用户问题的有效知识，那么答案很可能就是错误的。例如，作为旅游手册的知识库中有大量文段是 兰州拉面如何制作的菜谱、兰州拉面的产地、兰州有哪些特产等，只有一两条信息描述了附近拉面馆的地址。那么当用户询问“兰州拉面怎么走？”时，知识库检索到的信息可能只是兰州拉面的选材、调味、烹饪方面的信息，而唯独没有检索到前方50米处有一家兰州拉面馆。用户也没有办法获得有效的答案。
- 缺少对答案做兜底验证的机制，假设运气很好，志愿者不仅听懂了游客的问题，也正确查找到了附近最近的两家拉面馆的信息，但是志愿者的回答方式是“向北走200米就到了”。这有可能是一个正确的答案，但不是一个好的答案，实际考察过景区地形后我们可能会发现，志愿者北面是后海，你不太可能穿过湖面去一个地方。实际路径可能是：先向东走50十米，再向北走绕过后海，走到湖对面去，才能走到正确位置。那这个“向北走200米...”的回答，从导航的角度就不能算是准确了。

7.3.2 策略一：提升检索精确度

1）优化文本解析过程

在构建知识库的时候，我们首先需要正确的从文档中提取有效语料。因此，优化文本解析的过程往往对提升RAG的性能有很大帮助。例如，从网页中提取有效信息时，我们需要判断哪些部分应该被去掉（比如页眉页脚标签），哪些部分应该被保留（比如属于网页内容的表格标签）。

2）优化分片模式

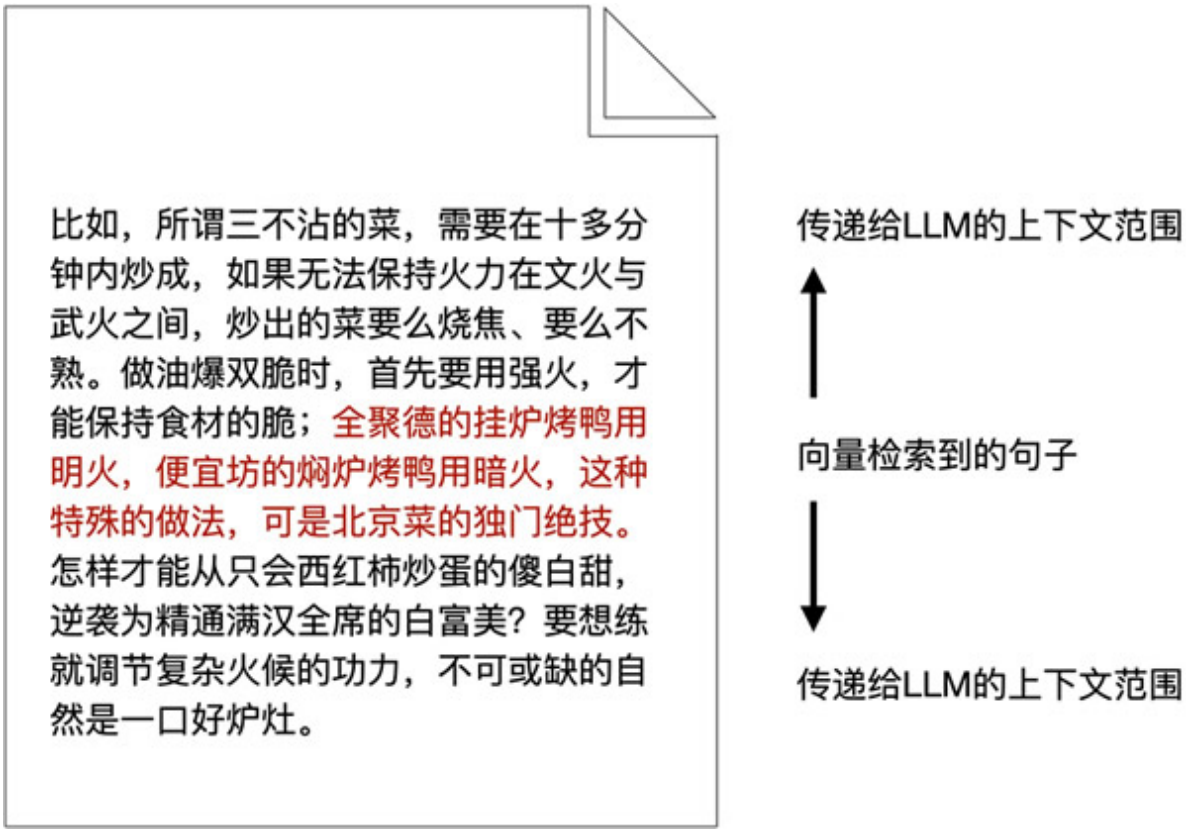
Chunk就是数据或信息的一个小片段或者区块。当你在处理大量的文本、数据或知识时，如果你一次性全部交给大模型来阅读和处理，效率是非常低的。所以，我们把它们切分成更小、更易管理的部分，这些部分就是chunk。每个chunk都包含了一些有用的信息，这样当系统根据用户的问题寻找某个知识时，它可以迅速定位到与答案相关信息的chunk，而不是在整个数据库中盲目搜索。因此，通过精心设计的chunk切分策略，系统能够更高效地检索信息，就像图书馆里按照类别和标签整理书籍一样，使得查找特定内容变得容易。优化chunk切分模式，就能加速信息检索、提升回答质量和生成效率。具体方法有很多：

- 利用领域知识：针对特定领域的文档，利用领域专有知识进行更精准的切分。例如，在法律文档中识别段落编号、条款作为切分依据。
- 基于固定大小切分：比如默认采用128个词或512个词切分为一个chunk，可以快速实现文本分块。缺点是忽略了语义和上下文完整性。

- 上下文感知：在切分时考虑前后文关系，避免信息断裂。可以通过保持特定句对或短语相邻，或使用更复杂的算法识别并保留语义完整性。最简单的做法是切分时保留前一句和后一句话。你也可以使用自然语言处理技术识别语义单元，如通过句子相似度计算、主题模型（如LDA）或BERT嵌入聚类来切分文本，确保每个chunk内部语义连贯，减少跨chunk信息依赖。通义实验室提供了一种文本切割模型，输入长文本即可得到切割好的文本块，详情可参考：中文文本分割模型。

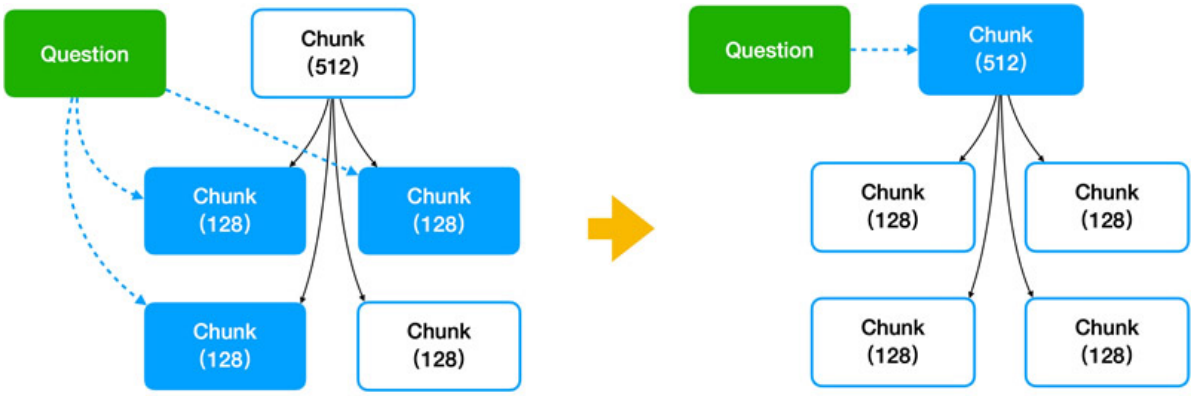
3）句子滑动窗口检索

这个策略是通过设置window_size（窗口大小）来调整提取句子的数量，当用户的问题匹配到一个chunk语料块时，通过窗函数提取目标语料块的上下文，而不仅仅是语料块本身，这样来获得更完整的语料上下文信息，提升RAG生成质量。



4）自动合并检索

这个策略是将文档分块，建成一棵语料块的树，比如1024切分、512切分、128切分，并构造出一棵文档结构树。当应用作搜索时，如果同一个父节点的多个叶子节点被选中，则返回整个父节点对应的语料块。从而确保与问题相关的语料信息被完整保留下来，从而大幅提升RAG的生成质量。实测发现这个方法比句子滑动窗口检索效果好。



对问题的检索匹配到了多个128切分的Chunks子节点，超过了预设值，因此采用512切分的父节点作为返回的Chunk

4）选择更适合业务的Embedding模型

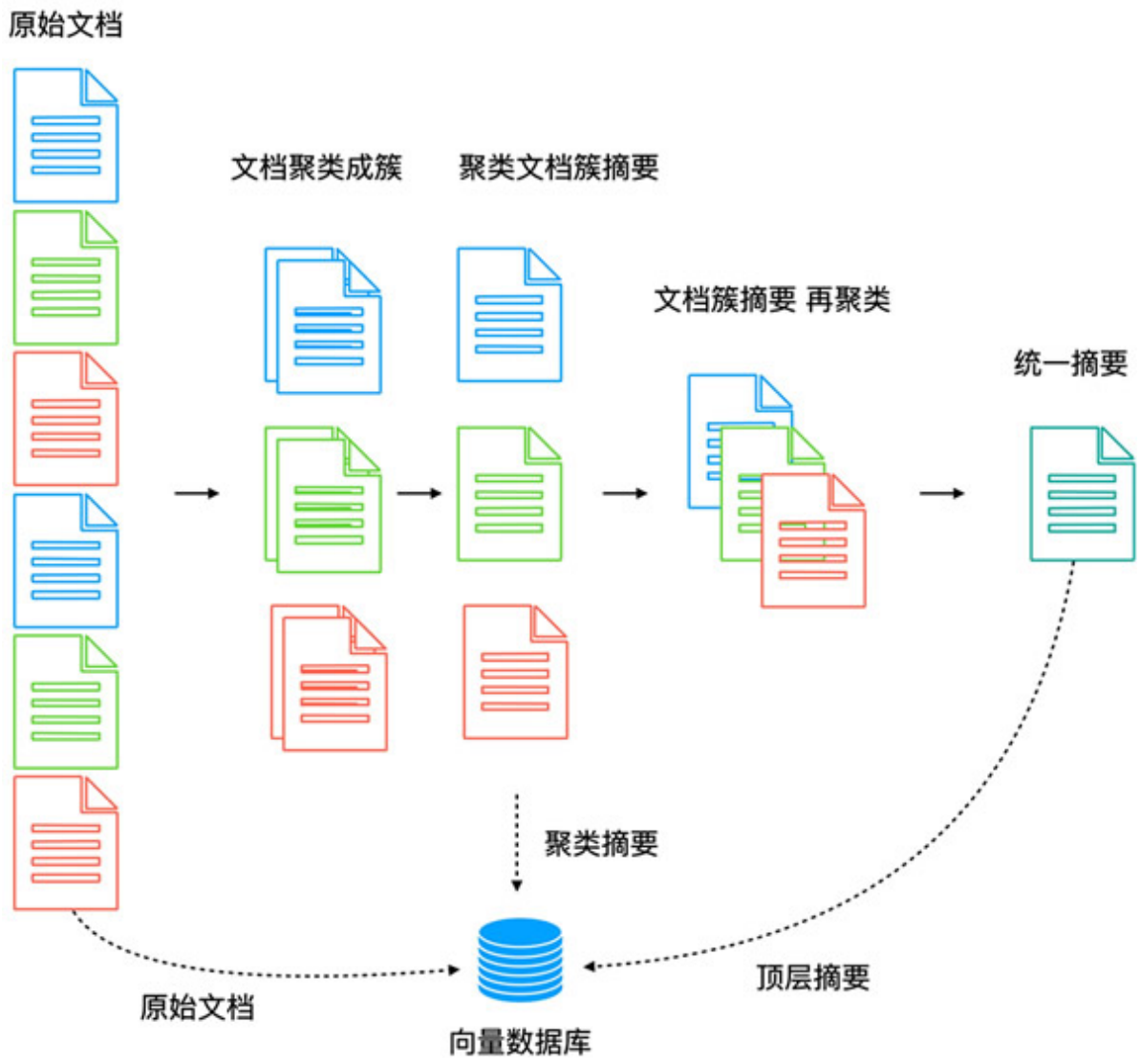
经过切分的语料块在提供检索服务之前，我们需要把chunk语料块由原来的文本内容转换为机器可以用于比对计算的一组数字，即变为Embedding向量。我们通过Embedding模型来进行这个转换。但是，由于不同的Embedding模型对于生成Embedding向量质量的影响很大，好的Embedding模型可以提升检索的准确率。 比如，针对中文检索的场景，我们应当选择在中文语料上表现更好的模型。那么针对你的业务场景，你也可以建议你的技术团队做Embedding模型的技术选型，挑选针对你的业务场景表现较好的模型。

5）选择更适合业务的ReRank模型

除了优化生成向量的质量，我们还需要同时优化做向量排序的ReRank模型，好的ReRank模型会让更贴近用户问题的chunks的排名更靠前。因此，我们也可以挑选能让你的业务应用表现更好的ReRank模型。

6）用聚类为文档块建立索引

还有一类做法是采用无监督聚类来生成文档索引。这就像通过文档的内容为文档自动建立目录的过程。假如志愿者拿到的文本资料是没有目录的，志愿者一页一页查找资料必然很慢。因此，可以将词条信息聚类，比如按照商店、公园、酒吧、咖啡店、中餐馆、快餐店等方式进行分组，建立目录，再根据汉语拼音字母来排序。这样志愿者来查找信息的时候就可以更快速地进行定位。

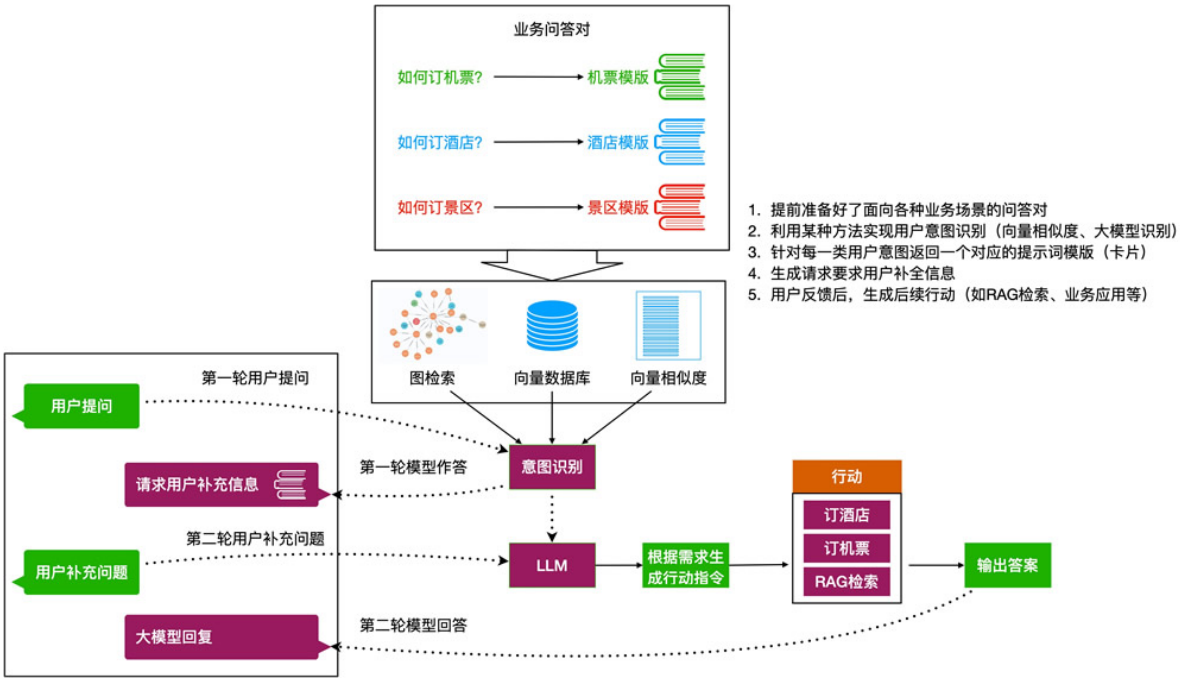


7.3.3 策略二：改进提问方式

我们希望做到能让人们通过口语对话来使用大模型应用。然而，人们在口语化表达自己的目的和意图时，往往会出现一些问题。比如，问题过于简单含糊出现了语义混淆，导致大模型理解错误；问题的要素非常多，而用户又讲得太少，只能在反复对话中不断沟通补全；问题涉及的知识点超出了大模型训练语料，或者知识库的覆盖范围，导致大模型编造了一些信息来回答等等。所以，我们期望能在用户提问的环节进行介入，让大模型能更好的理解用户的问题。

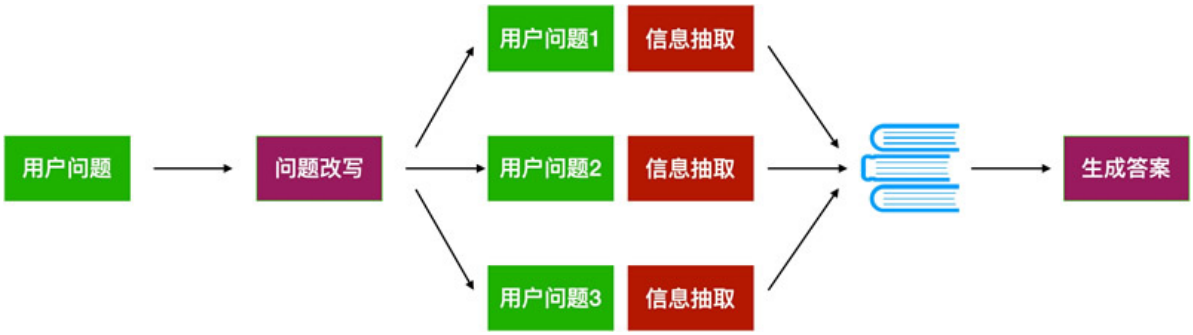
1）Enrich 完善用户问题

可以让大模型来完善用户的原始问题，产生一个更利于系统理解的完善后的用户问题，再让后续的系统去执行用户的需求。通过用大模型对用户的问题进行专业化改写，特别是加入了知识库的支持，可以生成更专业的问题。可以提供一套标准提示词模板，提供一些标准化的示例，也可以用知识库来增强。我们的主要目的是规范用户的输入请求，再生成RAG查询指令，从而提升RAG查询质量。



2）补全信息

有一些应用场景需要大量的参数支撑，（比如订火车票需要起点、终点、时间、座位等级、座位偏好等等），我们还可以进一步完善上面的思路，一次性告诉用户系统需要什么信息，让用户来补全。首先，需要准确理解用户的意图，实现意图识别的手段很多，如使用向量相似度匹配、使用搜索引擎、或者直接大模型来支持。其次，根据用户意图选择合适的业务需求模板。接着，让大模型参考业务需求模版来生成一段对话发给用户，请求用户补充信息。这时，如果用户进行了信息完善，那么大模型就可以基于用户的回复信息结合用户的请求来生成下一步的行动指令，整个系统就可以实现应用系统自动帮助用户订机票、订酒店，完成知识库问答等应用形式。



3）Multi-Query 多路召回

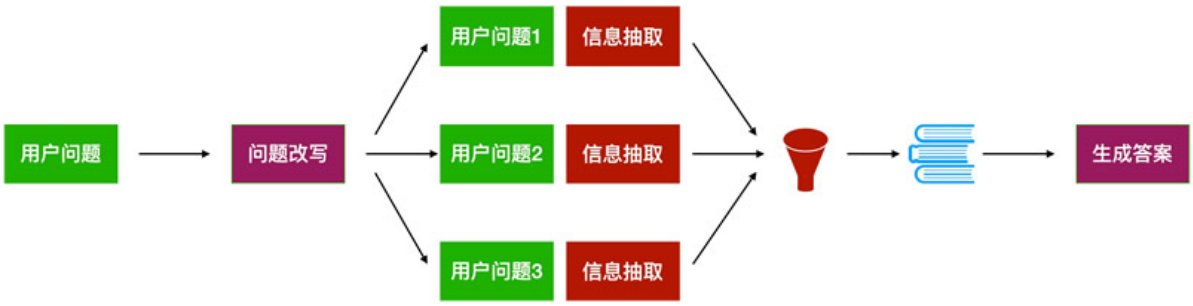
多路召回的方法不是让大模型进行一次改写然后反复向用户确认，而是让大模型自己解决如何理解用户的问题。所以我们首先一次性改写出多种用户问题，让大模型根据用户提出的问题，从多种不同角度去生成有一定提问角度或提问内容上存在差异的问题。让这些存在差异的问题作为大模型对用户真实需求的猜测，然后再把每个问题分别生成答案，并总结出最终答案。

例如：用户问“烤鸭店在哪里？”，大模型可能生成一系列近似问题：

```
附近有没有烤鸭店可以推荐?
北京著名的烤鸭店怎么走?
想吃烤鸭了，这附近哪里有卖的?
哪里能找到口味正宗的烤鸭店?
离我最近的烤鸭餐厅地址是多少?
```

4）RAG-Fusion 过滤融合

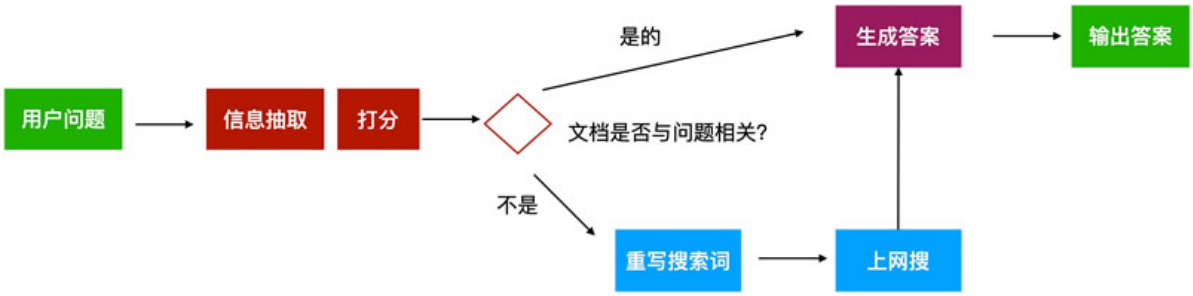
在经过多路召回获取了各种语料块之后，并不是将所有检索到的语料块都交给大模型，而是先进行一轮筛选，给检索到的语料块进行去重操作，然后按照与原问题的相关性进行排序，再将语料块打包喂给大模型来生成答案。经过去重复语料筛选，节省了传递给大模型的tokens数量，再经过排序，将更有价值的语料块传递给大模型，从而提升答案的生成质量。 用我们的例子讲就是，志愿者先从多种角度来理解用户的问题，然后对每个问题都去检索资料，查找有用信息，最后把重复信息去掉，再将获取的资料排序。这就能锁定比较接近用户问题的几段语料了，比如“全聚德烤鸭店的地址”，“天外天烤鸭店的地址”，“郭林家常菜店铺的地址”等等，以及这些烤鸭店分布在后海的那些区域，如何步行走过去等等。



7.3.4 策略三：改造检索渠道

Corrective Retrieval Augmented Generation (CRAG)是一种改善提取信息质量的策略：如果通过知识库检索得到的信息与用户问题相关性太低，我们就主动搜索互联网，将网络搜索到的信息与知识库搜索到的信息合并，再让大模型进行整理给出最终答案。在工程上我们可以有两种实现方式：

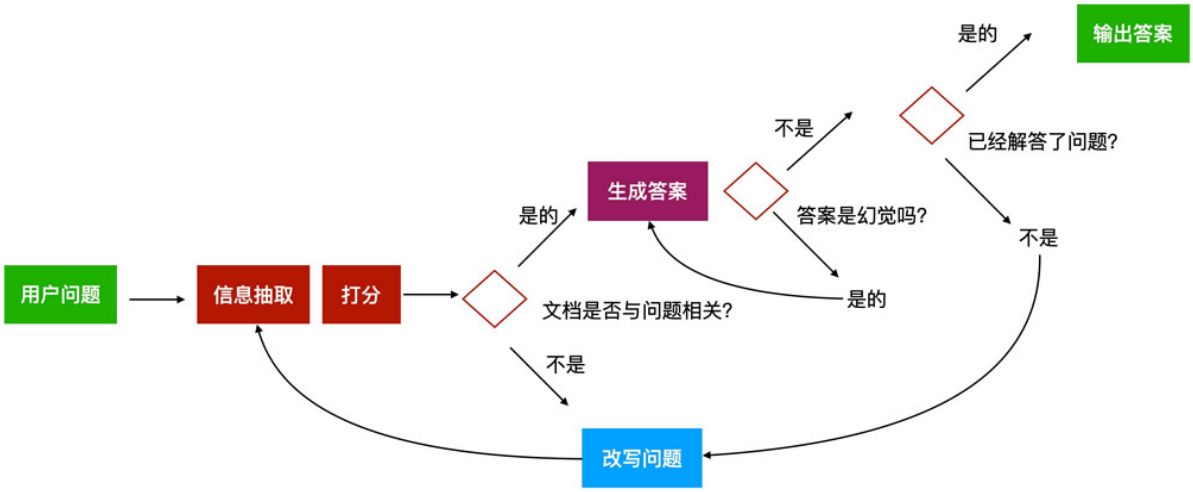
- 向量相似度，我们用检索信息得到的向量相似度分来判断。判断每个语料块与用户问题的相似度评分，是否高过某个阈值，如果搜索到的语料块与用户问题的相似度都比较低，就代表知识库中的信息与用户问题不太相关；
- 直接问大模型，我们可以先将知识库检索到的信息交给大模型，让大模型自主判断，这些资料是否能回答用户的问题。



7.3.5 策略四：回答前反复思考

Self-RAG，也称为self-reflection，是一种通过在应用中设计反馈路径实现自我反思的策略。基于这个思想，可以让应用问自己三个问题：

- 相关性：我获取的这些材料和问题相关吗？
- 无幻觉：我的答案是不是按照材料写的来讲，还是我自己编造的？
- 已解答：我的答案是不是解答了问题？



7.4 小结

RAG主要由两个部分构成：

- 建立索引：首先要清洗和提取原始数据，将 PDF、Docx等不同格式的文件解析为纯文本数据；然后将文本数据分割成更小的片段（chunk）；最后将这些片段经过嵌入模型转换成向量数据（此过程叫做embedding），并将原始语料块和嵌入向量以键值对形式存储到向量数据库中，以便进行后续快速且频繁的搜索。这就是建立索引的过程。
    - 文档加载，并按一定条件**切割**成片段
    - 将切割后的文档转化为 embedding
    - 把 embedding 存储到 embeddingStore 中
- 检索生成：系统会获取到用户输入，随后计算出用户的问题与向量数据库中的文档块之间的相似度，选择相似度最高的K个文档块（K值可以自己设置）作为回答当前问题的知识。知识与问题会合并到提示词模板中提交给大模型，大模型给出回复。这就是检索生成的过程。
    - 把用户 Query 转化为 embedding，queryEmbedding
    - 检索最相似的几个文档，找到最相似的 K 个
    - 取回 K 个最相似的文档文本
    - 把文本发送给大模型“包装”，生成最终返回给用户的文本